

北京理工大学

本科生毕业设计（论文）

LeRobot 平台感知—推理—执行流程的 系统级性能剖析研究

System-Level Performance Profiling of
Perception-Inference-Execution Pipeline
in the LeRobot Framework

学 院： 计算机学院

专 业： 软件工程

班 级： 08012204

学 生 姓 名： 俞乐楠

学 号： 1120221303

指 导 教 师： 王勇

2026 年 4 月 20 日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：_____ 日期：_____ 年 ____ 月 ____ 日

关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；学校可允许本毕业设计（论文）被查阅或借阅；学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：_____ 日期：_____ 年 ____ 月 ____ 日

指导教师签名：_____ 日期：_____ 年 ____ 月 ____ 日

LeRobot 平台感知—推理—执行流程的 系统级性能剖析研究

摘 要

本文以树莓派 5 (ARM Cortex-A76, 4 核, 4 GB LPDDR4X, 无 GPU) 为边缘推理平台, 对 HuggingFace LeRobot 框架下的 ACT 策略在 SO-101 六自由度机械臂上的在线感知—推理—执行循环进行系统级性能剖析。针对现有研究多聚焦单一层次、缺乏跨层次联合分析的不足, 尤其是主流优化往往集中于模型压缩、模型结构改进或硬件通信链路调优, 而相对忽略推理运行时、操作系统调度与设备 I/O 栈等基础软件因素的影响, 本文提出覆盖 AI 推理框架层、Python 运行时层、Linux 内核层与硬件 I/O 层的四层性能分析模型, 联合使用 ftrace、strace, 并在 pselect6 与 futex 路径上加入自定义 trace_printk 内核插桩, 端到端量化三阶段的时间分布, 定位主要瓶颈并系统梳理可行的优化路径。

工具链校准实验表明, strace 使主循环 FPS 下降 40.9%、上下文切换次数放大一个数量级, 不能用于定量时序分析; ftrace 延迟导出方案 FPS 仅下降 13.9%, 被选为正式实验的内核态工具。在 chunk_size=100、目标 30 FPS 配置下, 系统实测帧率仅约 18.7 FPS, 距目标存在约 38% 缺口。逐模块剖析显示, 单次 ACT 前向推理耗时约 1925 ms, 其中 ResNet18 视觉特征提取占 66.6%、Transformer Encoder 占 29.6%, 是绝对主瓶颈; 摄像头采用后台异步预取, 主线程感知耗时仅毫秒级, V4L2 阻塞与 MJPEG 解码主要由后台线程承担; 舵机 sync_read 约 1.2 ms、sync_write 约 1–3 ms, 而 write 系统调用本身仅约 38 μ s, 执行瓶颈在于 1 Mbps 波特率与半双工总线的物理约束, 而非软件或系统调用路径。

针对 ACT 推理这一主瓶颈, 本文进一步推导出异步推理稳态无 stall 的可行性不等式 $T_{\text{infer}}F \leq \min(hN, N/2)$, 并建立 FPS 随 hold 在临界点前上升、临界点后饱和到 30 FPS 的分段模型。通过 hold 参数 0.10–0.60 的扫描实验, 本文在动作质量可接受前提下确定 hold=0.30 为推荐默认值, 系统有效控制频率达 27.42 FPS, 相比同步基线 16.13 FPS 提升 70.0%。结果表明, 异步调度能够显著回收动作执行期间的空窗; 实测 episode 平均值未完全达到 30 FPS, 主要来自启动空队列等待和树莓派 5 单机部署下的 CPU 资源争抢。未来仍需结合 INT8 量化、ONNX Runtime 等手段降低有效返回时延, 并通过调度隔离减少资源争抢, 才能从根本上稳定到达 30 FPS 控制目标。

关键词： LeRobot; ACT 策略; 性能剖析; ARM 推理; ftrace; chunk_size; 推理异步化

System-Level Performance Profiling of Perception-Inference-Execution Pipeline in the LeRobot Framework

Abstract

This paper presents a system-level performance analysis of the ACT (Action Chunking with Transformers) policy running online on a Raspberry Pi 5 (ARM Cortex-A76, 4 cores, 4 GB LPDDR4X, no GPU) within HuggingFace’s LeRobot framework, controlling an SO-101 6-DoF arm. Prior optimization efforts often focus on model compression, architecture changes, or hardware communication links, while underestimating the impact of inference runtimes, OS scheduling, and device I/O stacks. To address this lack of cross-layer joint analysis, we propose a four-layer profiling model spanning the AI inference framework, Python runtime, Linux kernel, and hardware I/O layers, and combine Python logging, torch.profiler, ftrace, strace, and custom `trace_printk` instrumentation in the kernel `pselect6` and `futex` paths to quantify the end-to-end time distribution across perception, inference, and execution, locate the dominant bottleneck, and systematically review feasible optimization paths.

A tool-calibration experiment shows that strace amplifies context switches by roughly an order of magnitude and reduces FPS by 40.9%, ruling it out for quantitative use; ftrace with deferred dump incurs only a 13.9% FPS drop and is adopted as the kernel-side tool. At `chunk_size=100` with a 30 FPS target, the system reaches only ≈ 18.7 FPS, leaving a 38% gap. Per-module breakdown shows that a single ACT forward pass takes about 1925 ms, with ResNet18 visual feature extraction accounting for 66.6% and the Transformer encoder for 29.6% — the absolute bottleneck. Camera frames are prefetched in a background thread, so perception cost on the main thread is in the millisecond range; servo `sync_read` costs ≈ 1.2 ms and `sync_write` 1–3 ms, while the underlying `write` syscall itself takes only $\approx 38 \mu\text{s}$, indicating that the execution bottleneck lies in the 1 Mbps half-duplex bus rather than software or syscall paths.

To recover idle CPU windows during inference, we derive a feasibility inequality

for asynchronous inference, $T_{\text{infer}}F \leq \min(hN, N/2)$, and model the FPS–hold curve as rising before a critical hold and saturating at 30 FPS afterwards. A hold sweep from 0.10 to 0.60 shows that, with acceptable action quality, hold = 0.30 is recommended as the default and yields an effective control rate of 27.42 FPS, a 70.0% improvement over the synchronous 16.13 FPS baseline. The remaining gap to 30 FPS mainly comes from startup queue starvation and CPU contention between the client, server, camera, and serial paths on Raspberry Pi 5. Future work on INT8 quantization, ONNX Runtime, and scheduling isolation is needed to fully achieve a stable 30 FPS control target.

Key Words: LeRobot; ACT policy; performance profiling; ARM inference; ftrace; chunk_size; asynchronous inference

目录

第一章 引言	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.3 研究目标	3
第二章 实验环境构建	4
2.1 实验平台选型	4
2.2 推理循环与软件栈	6
2.3 实验任务与数据集	8
第三章 性能分析	9
3.1 工具链选型	10
3.2 感知阶段性能剖析	14
3.2.1 摄像头异步读取机制	14
3.2.2 是否进入内核	15
3.2.3 OpenCV 到内核的边界	16
3.2.4 时间剖析	17
3.3 推理阶段性能剖析	17
3.3.1 <code>select_action</code> 与动作队列	17
3.3.2 一次完整 ACT 推理的数据流	18
3.3.3 时间剖析: <code>chunk_size</code> 参数扫描	19
3.3.4 单次 ACT 推理耗时分解	21
3.3.5 PyTorch 线程行为与推理优化研究现状	22
3.4 执行阶段性能剖析	22
3.4.1 LeRobot 执行路径软件栈	22
3.4.2 <code>sync_write</code> 写链路: 从电机总线到串口写入	23
3.4.3 <code>sync_read</code> 读链路: 从请求帧到逐电机轮询回包	23
第四章 优化	25
4.1 异步化路径选择	25
4.2 推理异步化	26

4.2.1	设计思路	26
4.2.2	实现结构	26
4.2.3	可行性不等式与 hold 参数	27
4.2.4	不等式在本平台的代入	31
4.2.5	FPS 与 hold 的定量关系	32
4.2.6	树莓派 5 单机部署的 CPU 影响	33
4.3	异步推理收益的实验验证	33
4.3.1	实验目的与论证逻辑	33
4.3.2	实验配置与数据来源	34
4.3.3	打点与对齐方法	34
4.3.4	实验结果	35
4.3.5	结果讨论	36
第五章 总结与展望		38
5.1	主要工作总结	38
5.2	研究局限性	39
5.3	未来工作	39
参考文献		41
附录		43
致谢		44

第一章 引言

1.1 研究背景与意义

近年来,协作机器人与自主移动机器人在工业制造、仓储物流、医疗服务等领域加速落地,成为推动产业智能化转型的核心载体。视觉—语言—动作 (Vision-Language-Action, VLA) 大模型将视觉感知与动作决策统一到端到端神经网络中,显著拓展了机器人在开放场景下的泛化能力。国家“十五五”规划 (2026—2030 年) 明确将具身智能机器人列为战略性新兴产业重点方向,标志着机器人从辅助性工具向智能制造和数字经济核心基础设施的历史性跃升。在此背景下,如何在资源受限的边缘平台上高效、可靠地运行机器人智能软件,已成为制约机器人大规模部署的关键问题。

从技术栈角度看,机器人系统通常由三个层次组成。**硬件层**涵盖上位机 (嵌入式计算板)、传感器 (摄像头、力传感器)、执行器 (电机、舵机) 与机械结构,决定机器人的物理感知与运动能力;**算法层**包括视觉感知、策略规划与运动控制,以 ACT^[1]、Diffusion Policy^[2] 等端到端模仿学习模型为代表;**基础软件层**则由操作系统 (Linux 等)、通信中间件 (ROS 等) 和推理框架 (PyTorch、ONNX Runtime、TFLite 等) 构成,向下管理硬件资源,向上支撑算法模型的运行。本文的研究聚焦于基础软件层:操作系统的内核调度策略、系统调用路径与设备驱动模型,以及推理框架的线程管理与算子调度,共同决定着机器人系统的硬件生态兼容性、推理性能、决策准确性、运行可靠性和系统安全性,是整个机器人软件栈性能与可信度的基础。

面向机器人基础软件的研究已有大量探索,但仍存在明显不足。广义上,机器人基础软件涵盖三个方向:操作系统内核、推理库、以及专用于机器人的通信工具。

在**操作系统内核**层面,现有路线可归纳为三类:以 Linux 为代表的**宏内核系统**应用广泛,但系统调用需经历用户态—内核态切换,单次操作引入百纳秒至数微秒的不确定延迟;以 HarmonyOS、Fuchsia 为代表的**微内核系统**结构精简、隔离性强,但驱动驻于用户态,硬件操作需经历两次态切换,整体延迟往往反而高于宏内核;以 FreeRTOS、Zephyr 为代表的**极简嵌入式实时系统**调度开销极低,但缺乏完整的深度学习推理支持,难以直接承载现代策略模型。

在**推理库**层面,PyTorch、ONNX Runtime、TFLite 各有侧重,均以“外挂式”方式接入操作系统:其内置线程池 (OpenMP 或 C10) 的调度与 OS 内核 CFS 调度器相互独立,在资源受限平台上容易引发调度冲突,造成额外的上下文切换与 futex 等待开销。

在**机器人专用通信工具**层面，ROS/ROS2 以 DDS 为底层通信层，micro-ROS 则将其下沉至嵌入式 MCU，二者均简化了多节点协作开发，但本质上继承了底层内核的调度局限，节点间消息传递延迟受制于 Linux CFS 调度器，无硬实时保证。

更根本的问题在于，现有性能优化与分析工作多聚焦于单一层次。主流优化路径往往集中在模型压缩、模型结构改进或硬件通信链路调优上，而对推理运行时、Python 线程行为、OS 内核调度、系统调用阻塞与设备 I/O 栈等基础软件因素关注不足。这使得许多工作虽然能够报告端到端延迟或单点优化收益，却缺乏从 AI 推理框架层到 OS 内核层的跨层次联合剖析，难以回答“瓶颈究竟在哪一层、由什么原因导致”等关键问题。

针对上述研究缺口，本文以 HuggingFace LeRobot 框架^[3]下 ACT 策略在树莓派 5（ARM Cortex-A76，4 核，4 GB LPDDR4X，无 GPU）上的在线推理为研究对象，系统开展机器人基础软件的跨层次性能剖析。实验表明，单次 ACT 完整推理耗时 1924.8 ms，其中 ResNet18 视觉编码器占 66.6%（1281 ms）为第一瓶颈；在 `chunk_size = 100` 的同步推理配置下实测帧率仅 18.7 fps，距 30 fps 目标仍有约 38% 缺口。通过推理异步化优化（`hold=0.30`），有效帧率提升至 27.42 fps（相对提升 70%），基本接近实时目标。本文提出覆盖 AI 推理框架层、Python 运行时层、OS 内核层和硬件 I/O 层的四层性能分析模型，建立面向“感知—推理—执行”紧耦合场景的多工具联合剖析方法，通过 `torch.profiler`/`ftrace`/`perf` 多工具联合剖析，端到端量化各阶段时间开销，定位 ResNet18 编码器为第一瓶颈（占 66.6%），揭示 `strace` 带来 40.9% FPS 下降，验证 `ftrace` 的低扰动必要性，为嵌入式机器人基础软件的性能优化提供明确的优先级依据。

1.2 国内外研究现状

机器人基础软件层面的研究可大致归为三个方向：操作系统、通信构件与开发 SDK；本节按此三个方向梳理国内外现有工作，并指出研究空白。

在操作系统层面，研究核心是**实时性保证**。标准 Linux 内核基于 CFS 公平调度，不提供硬实时保证；PREEMPT-RT 补丁通过把内核大段代码改为可抢占，将 ARM 嵌入式平台上的网络通信最坏延迟从 vanilla Linux 的 13197 μ s 压降至 88–110 μ s^[4]，是目前机器人控制系统最主流的 Linux 实时化方案；但仅有 PREEMPT-RT 还不够，并发流量下仍需 `cpusets` CPU 绑核隔离才能维持有界延迟^[4]。ROS2 实时性研究综述^[5]进一步指出，内核 Executor 与定制调度器是该方向密集投入的研究分支。近期 AMS^[6]（Zheng et al., 2025）将 action context、action exception、action replay 三个

OS 原语融入 VLA 推理管理，在真实机械臂任务上将成功率提升 $7\times-24\times$ ，执行步数减少 29%-74%，表明 OS 层机制对机器人推理的性能与可靠性具有直接影响。

在通信构件层面，ROS2^[7] 以 DDS (Data Distribution Service) 为底层通信层，是学术界与工业界的事实标准；但其节点调度仍依赖 Linux CFS，无硬实时保证。针对 ARM 嵌入式平台的实测表明，节点组合方式 (intra-process vs. multi-process) 对 CPU 占用与消息延迟有显著影响^[8]；在树莓派 3/4 等平台上，Linux 网络栈被发现是制约硬实时以太网通信的主要瓶颈^[9]。为支撑此类分析，ros2_tracing^[10] 提供了基于 LTTng 的低开销追踪框架，可在不显著扰动主循环的条件下获取 ROS2 节点调度与消息传递的精细时序。

在 SDK 层面，深度学习推理框架是机器人基础软件最核心的组成之一。TFLite^[11] 面向 MCU 级极端受限场景，支持 INT8 量化与 ARM NEON SIMD 加速；ONNX Runtime^[12] 通过 KleidiAI 算子库在 ARM64 上实现 28-51% 的性能提升；PyTorch 则提供 TorchScript 与 CPU 优化，是 LeRobot^[3] 等机器人开源框架的默认推理后端。在机器人生态软件方面，HuggingFace LeRobot 等以 Python 为主的开源框架快速降低了 ACT^[1]、Diffusion Policy^[2] 等策略的训练与部署门槛，但围绕其性能特性的研究仍以模型精度与吞吐量为主要评估维度。更关键的是，上述推理框架与生态软件均以“外挂式”方式接入操作系统：其内置线程池 (OpenMP 或 C10) 的调度与 OS 内核 CFS 调度器相互独立，在资源受限平台上容易引发调度冲突，引入额外的 futex 阻塞与上下文切换开销，而这一类内核层耦合开销迄今缺乏系统性量化。

综上，已有工作多聚焦于上述三个层面中的**单一层面**：或针对内核做实时化改造、或评估通信中间件的延迟、或优化推理框架的算子性能；即便涉及多层（如 ros2_tracing 联合用户态—内核态追踪），其覆盖范围也仅限于 ROS2 通信链路。**从 Python 运行时到 AI 推理框架再到 Linux 内核**这条贯穿机器人推理主循环的端到端调用链，迄今仍缺乏系统性的跨层次联合剖析。本文的研究方向正是**跨层次联合延迟分析**：建立覆盖 AI 推理框架层、Python 运行时层、OS 内核层与硬件 I/O 层的四层性能模型，通过 ftrace 等工具联合剖析，端到端定位 ARM 嵌入式平台上机器人推理主循环的真实瓶颈所在。

1.3 研究目标

本文以 LeRobot 为代表的机器人软硬件栈为研究对象，构建覆盖 AI 推理框架层、Python 运行时层、OS 内核层与硬件 I/O 层的跨层次性能分析框架，弥补现有工作偏重模型或硬件通信单点优化、较少量化基础软件影响的不足，发现各阶段性能

瓶颈，进行系统性量化测量，为后续性能优化提供实验依据。

第二章 实验环境构建

2.1 实验平台选型

本文选用 LeRobot^[3]——HuggingFace 推出的开源机器人学习库，官方定位为“Making AI for Robotics more accessible with end-to-end learning”，提供从数据采集、模型训练到在线部署的完整工具链，支持 ACT、Diffusion Policy 等模仿学习策略。LeRobot 以单进程 Python 脚本串行执行感知—推理—执行三阶段，无分布式依赖；相比之下，基于 ROS2^[7] 的方案以 DDS 为底层通信中间件，单次跨节点消息传递引入 1–5 ms 延迟，DDS 守护进程还额外占用 CPU 与内存^[8]；第三章的剖析将表明通信开销远低于推理开销，引入 ROS2 只会带来不必要的测量干扰。

推理主机选用树莓派 5（ARM Cortex-A76，4 核，4 GB LPDDR4X，无 GPU），搭配 SO-101 六自由度机械臂与两路 USB 摄像头。SO-101 是 LeRobot 配套的低成本开源机械臂，结构件可 3D 打印，具备完整的遥操作与部署支持。实验搭建了两个 SO-101 follower，每个含 6 个舵机，由一块驱动板统一控制；在线推理实验只使用其中一个 follower，两路摄像头（手眼与固定）均通过 OpenCV 采集。

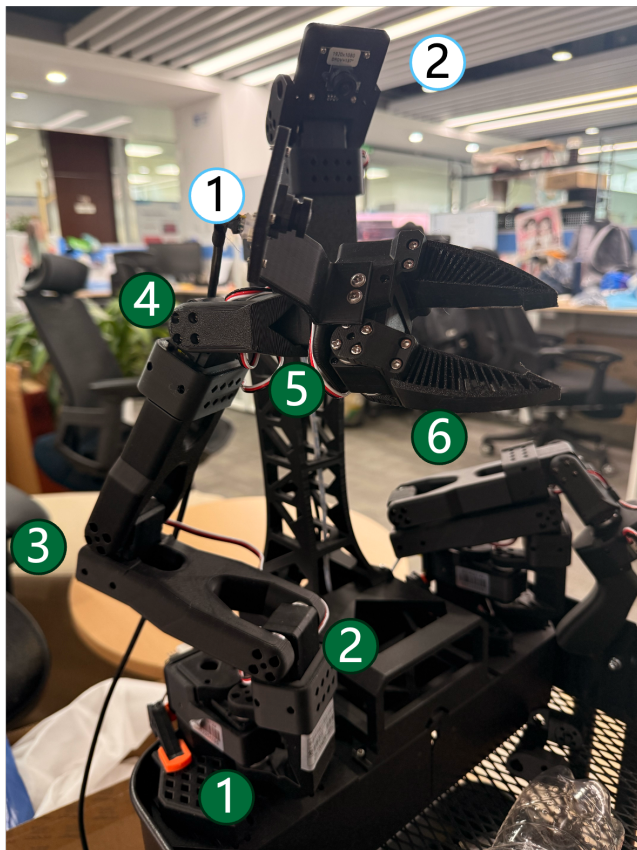


图 1: SO-101 六自由度机械臂实物（标注关节编号 1-6）

编号	名称	说明
1	shoulder_pan	底座水平旋转关节
2	shoulder_lift	肩部俯仰关节
3	elbow_flex	肘部弯曲关节
4	wrist_flex	腕部弯曲关节
5	wrist_roll	腕部滚转关节
6	gripper	夹爪（末端执行器）
C1	handeye camera	手眼相机，随臂运动，采集末端视角图像
C2	fixed camera	固定相机，俯视工作台，采集场景全局图像

策略模型选用 ACT^[1] (Action Chunking with Transformers)，以 ResNet18 + Transformer 构成约 50M 参数的端到端模仿学习策略。以 RT-2^[13]、OpenVLA^[14] 为代表的 VLA 模型参数量达 7B+，在无 GPU 的 ARM 平台上几乎无法实时运行；Diffusion Policy^[2] 参数量相近但扩散推理步数多导致延迟更高；ACT 是本文研究场景下可在树莓派 5 上实时运行的唯一可行选型。

项目	配置
推理主机	树莓派 5, ARM Cortex-A76, 4 核, 4 GB LPDDR4X, 无 GPU
机械臂	SO-101 follower ×2; 推理时使用其中 1 个 follower
舵机与驱动	每个 follower 含 6 个舵机, 由 1 块舵机驱动板控制
摄像头	USB 摄像头 ×2, 位置分别为 handeye 和 fixed
图像采集	OpenCV Camera, 分辨率 640×360, 30 fps
控制参数	chunk_size =100, 控制目标 fps=30, 串口波特率 1 Mbps
采集设置	每类任务采集 ≥ 3 次 episode 取均值, 排除冷启动影响
软件/库	版本或说明
操作系统	Raspberry Pi OS
Python	3.10
LeRobot	0.3.4
PyTorch	2.7.1
torchvision	0.22.1
OpenCV	4.12.0.88
NumPy	2.2.6
pyserial	3.5
feetech-servo-sdk	1.0.0

2.2 推理循环与软件栈

LeRobot 的在线推理是一个持续运行的帧循环: 每一帧依次经历**观测采集**(`get_observation()`)、**策略推理**(`select_action()`)、**指令下发**(`send_action()`)和**等待**四个阶段, 周而复始直到 episode 结束 (图 2)。该循环在单个 Python 进程内串行执行, 不依赖外部消息中间件, 若以 30 fps 作为控制目标, 则单帧可用时间约为 33.3 ms。

每个阶段调用不同的软件栈路径 (图 3): **观测阶段**同时采集摄像头图像和关节角度——摄像头图像通过 OpenCV 后台线程异步预取, 主线程调用 `async_read()` 取回已解码帧, 底层经由 V4L2 驱动和内核 `pselect6` 等待帧就绪; 关节角度由 `scservo_sdk` 通过 `pyserial` 以 `sync_read` 指令向舵机总线发起同步读取, 底层经由串口驱动和内核 `read/write` 系统调用完成半双工通信。**推理阶段**将观测张量送入

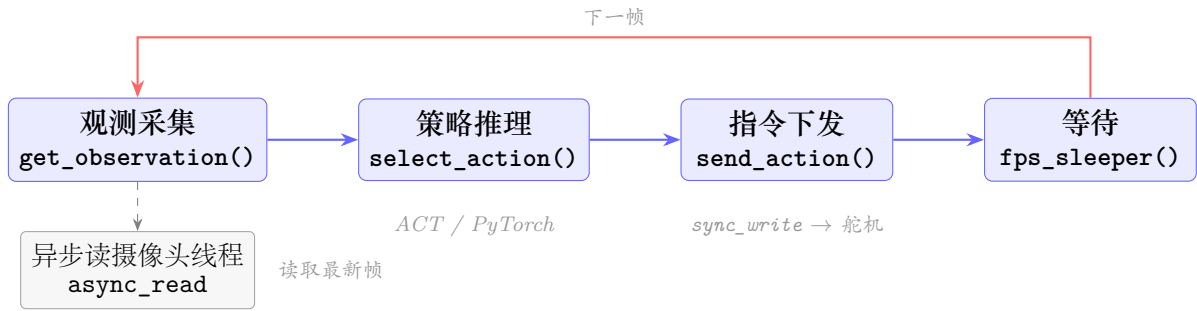


图 2: LeRobot 单帧推理循环流程：观测 → 推理 → 执行 → 等待，周而复始

ACT 策略网络，由 PyTorch（C10 线程池 + ARM NEON 算子）在 CPU 上完成前向计算，经由 glibc 和 Linux 内核的 futex 机制协调线程同步。**执行阶段**将预测动作通过 scservo_sdk 以 `sync_write` 写回舵机，路径与观测阶段的关节读取对称。**等待阶段**由 `fps_sleeper` 调用 `time.sleep()` 补足帧间隔，底层触发 `nanosleep` 系统调用使进程主动让出 CPU。

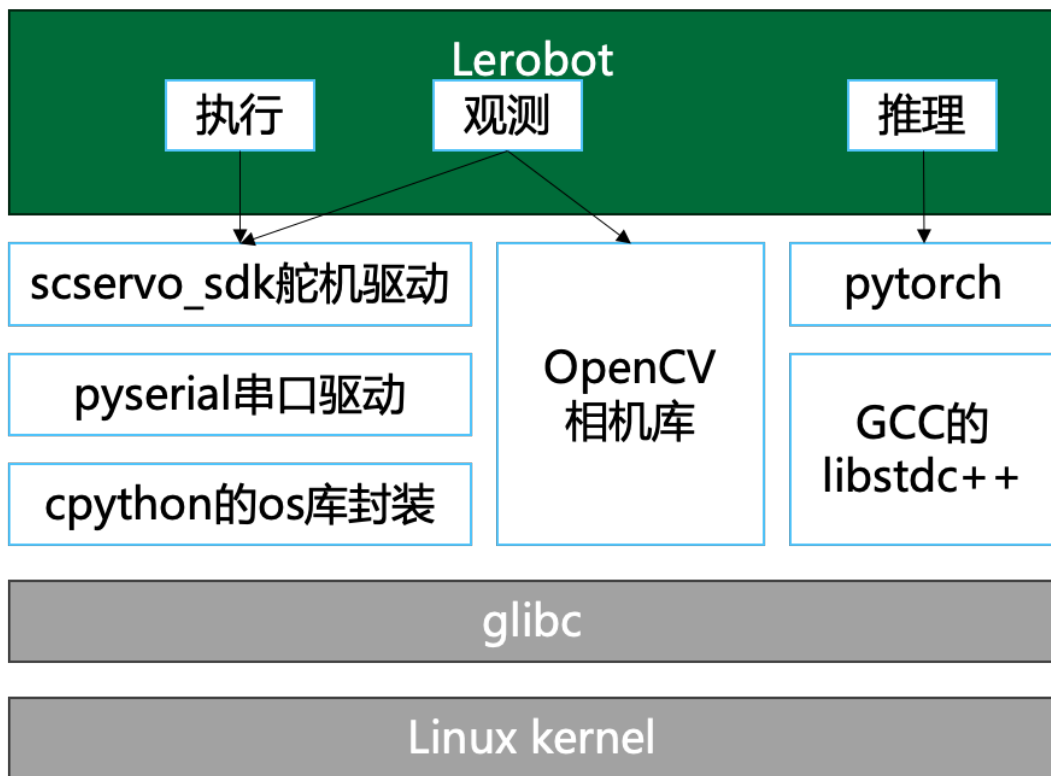


图 3: LeRobot 软件栈四层架构：应用逻辑层 → 库与驱动层 → glibc → Linux 内核

[来源：LeRobot 软件栈全链路分析.md]

2.3 实验任务与数据集

本文采用”遥操作采集—服务器训练—边缘推理”的标准流程构建实验数据集。数据采集阶段，操作者手持 leader 臂进行示教，SO-101 follower 臂实时同步复现动作，LeRobot 同步记录所有关节角度与两路摄像头图像，形成结构化 episode 文件。每类任务录制 50 个 episode，三类共 150 个 episode。训练阶段，将数据集传至配备 GPU 的服务器，调用 LeRobot `train` 命令以 ACT 策略进行监督学习，生成模型权重文件。推理阶段，将权重部署至树莓派 5，由 `record.py` 加载 policy 执行在线闭环推理，实验数据均在此阶段采集。

三类任务按操作复杂度从低到高设计，以考察不同难度下系统的推理行为是否存在差异：

难度	任务	描述
低	抓取 (Grasp)	场景中放置 1 个瓶子，机械臂识别并完成抓取；目标唯一，动作路径固定
中	推倒 (Push)	场景中放置 2 个瓶子，需识别指定目标并将其推倒；引入目标选择判断
高	分类 (Sort)	场景中放置 3 个瓶子，需按指定顺序逐一抓取并分类摆放；需要长程规划与多步序列操作

三类任务的场景布置与目标动作如图 4 所示。



图 4: 三类典型工作负载示意图：抓取、推倒、分类

三类任务统一使用树莓派 5、ACT (`chunk_size=100`)、目标帧率 30 fps，每类任务取 $n = 50$ 个 episode 的统计结果作为基准画像，只比较各阶段时间占比，而不把 episode 绝对时长差异混入分析。

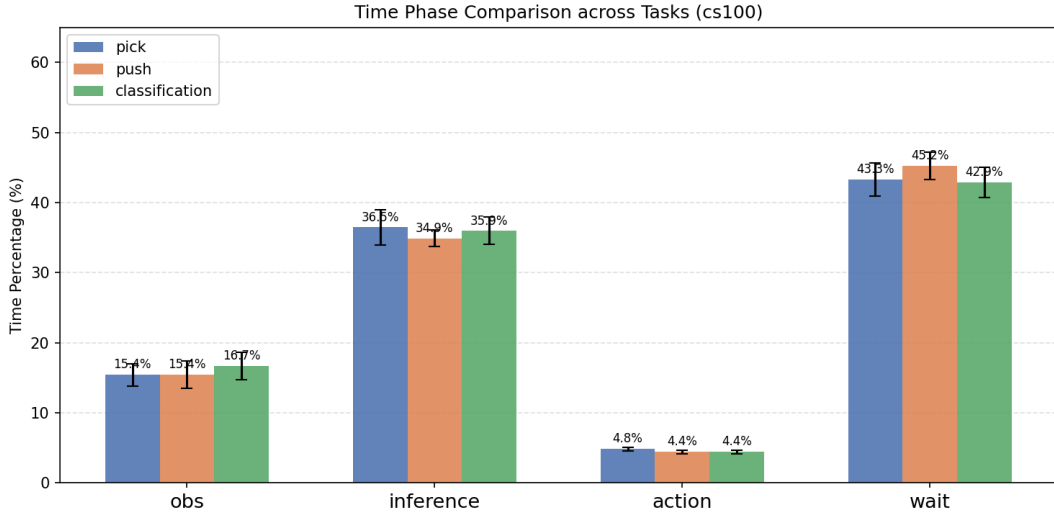


图 5: 三类任务在 `chunk_size=100` 下的阶段时间占比对比

图 5 的主要现象有三点。第一，三类任务的推理占比都集中在 35%–36% 左右，说明在模型结构、`chunk_size` 和输入配置相同的情况下，ACT 单次推理开销主要由模型本身决定，而不是由抓取、推倒、分类这些任务语义决定。第二，推倒任务的 `wait_pct` 最高，达到 45.2%，比抓取和分类高约 2 个百分点；这更像是动作行程和执行节奏造成的等待增加，而不是推理侧变慢。第三，`obs_pct` 和 `action_pct` 在三类任务之间变化很小，说明相机读取、舵机状态读取以及舵机写入在当前实验设置下更接近固定开销。

因此，后续性能分析不把三类任务视为三种完全不同的软件路径，而是把它们看作同一 LeRobot 推理循环在不同动作负载下的表现。抓取和分类更接近基准工作负载；推倒任务由于等待占比最高，更适合用来观察执行节奏和 `chunk_size` 对整体帧率的影响。

[来源：实验/02_ 三类任务工作负载对比/cs100_task_comparison.md]

第三章 性能分析

本章围绕 LeRobot 在线推理循环中的三个主要阶段展开：感知阶段负责采集两路相机图像和机械臂关节状态，推理阶段负责 ACT 策略前向计算，执行阶段负责将动作通过舵机总线下发到 SO-101 follower。三者虽然在 Python 主循环中串行出现，但其底层开销来源并不相同：摄像头路径主要经过 OpenCV/V4L2 和用户态图像解码，推理路径主要受 ARM CPU 上 PyTorch 算子计算限制，舵机路径则受半双工串

口总线和 SCServo 协议约束。

3.1 工具链选型

strace 与 ftrace 机制对比：

两者作用层次不同，开销来源也不同：

- **strace**：通过 `ptrace` 系统调用在用户态拦截目标进程的每一次系统调用入口与出口。每次拦截都需要一次进程上下文切换（目标进程 $\rightarrow$ strace 进程），其开销与系统调用频率正相关，频繁调用时对主循环干扰极大，不能作为定量时序数据来源（具体扰动量级见后文实测）。
- **ftrace**：通过 Linux 内核 tracepoint 在内核态直接写入 ring buffer，插桩本身开销极低（纳秒级，无进程切换）。但将 ring buffer 内容导出至用户态文件（`cat /sys/kernel/debug/tracing/trace > log` 或后台进程持续读取 `trace_pipe`）是一次批量/持续文件 I/O，当 event 密集时，导出过程本身会消耗额外 CPU 和内存带宽，且实时导出期间 ring buffer 也可能覆盖旧数据（丢失事件）。

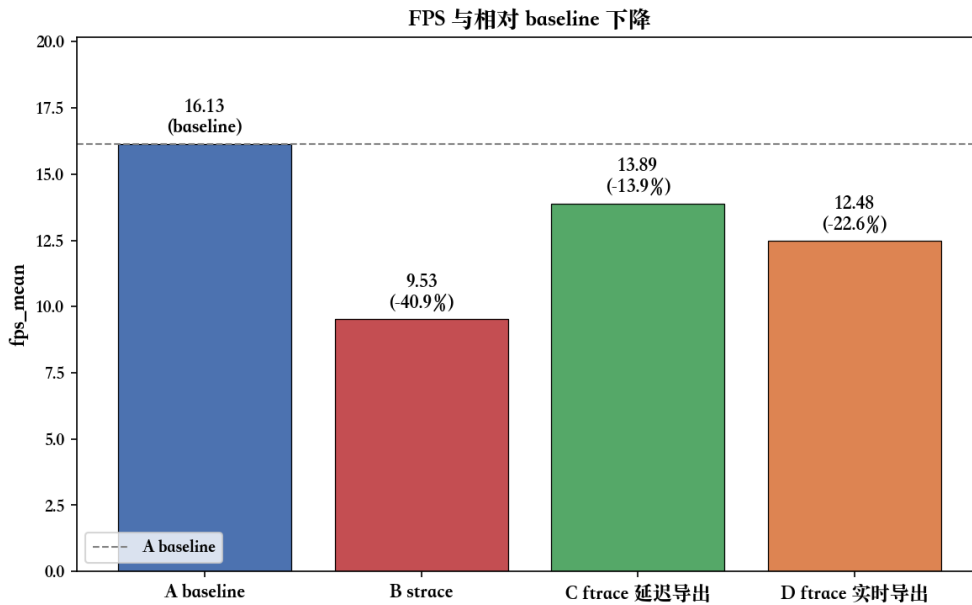
测量工具开销实测

为定量比较各工具对主循环的扰动，本文设计了一个测量工具校准实验。在固定 `chunk_size=100`、目标 `fps=30`、episode 时长约 30 s 的同一条 `record_loop` 上，对比四组方案：A 组 baseline 不开任何 tracing；B 组用 `strace -f -ttT` 包住 `record` 命令；C 组开启 `ftrace` 仅写内核 ring buffer，episode 结束后再导出 `trace`；D 组开启 `ftrace` 并同时启动后台进程实时读取 `trace_pipe` 写入文件。所有指标由 `record.py` 内部通过 `resource.getrusage()` 与 `sysfs` 自动采集，写入 `analysis/timing_stats.csv`。表 1 汇总了四组的核心指标，后文分别从 FPS、上下文切换次数和阶段占比漂移三个角度逐一分析。

表 1: 四组测量工具对 `record_loop` 的扰动（每组 $n = 1$ ，仅用于工具选型）

指标	A baseline	B strace	C ftrace 延迟	D ftrace 实时
<code>fps_mean</code>	16.13	9.53	13.89	12.48
相对 A 的 FPS 下降	—	-40.9%	-13.9%	-22.6%
<code>system_time_s</code>	1.40	2.76	1.37	1.03
<code>voluntary_cs</code>	23,947	750,662	30,377	36,336
<code>involuntary_cs</code>	78,436	541,334	84,481	52,907
<code>obs_pct</code> 漂移 (pp)	0	+34.8	-5.5	-3.0
<code>wait_pct</code> 漂移 (pp)	0	-31.6	-0.9	-0.8
trace 文件大小	—	157 MB	168 MB	823 MB

FPS 下降 图 6 给出了四组的实测 `fps_mean` 与相对 A 组 baseline 的下降幅度。B strace 组从 16.13 fps 跌至 9.53 fps，下降 40.9%，单这一项就足以排除 strace 作为正式时序数据来源的可能性。C ftrace 延迟导出组下降 13.9%，D ftrace 实时导出组下降 22.6%，两者都仍然超过设计文档定义的 $\leq 5\%$ 验收阈值；但需要注意每组只有 1 个 episode，A 组 baseline 自身存在抖动（如观测阶段单次冷启动时延变长会直接拉低绝对 FPS），正式实验需补足每组 5 个 episode 后再核对方差。即便如此，C 组优于 D 组、且两组都明显优于 B 组的相对秩序非常稳定，能够支撑后续的工具选型。

图 6: 四组测量工具下的 `fps_mean` 与相对 A 组 baseline 的下降百分比

上下文切换次数 图 7 用对数坐标对比了四组的自愿与非自愿上下文切换次数。B strace 组的 `voluntary_cs` 从 baseline 的约 2.4 万次飙升至约 75 万次（约 31 倍），`involuntary_cs` 也从 7.8 万次升至 54 万次（约 7 倍）。这种几何级放大是 `ptrace` 机制的直接体现：每一次系统调用都被强制中断到 strace 进程读取参数后再恢复，因此摄像头链路中密集出现的 `pselect6`、`read`、`ioctl` 都会在切换次数上留下印记。相比之下，C 组的切换次数仅比 baseline 高约 27%，D 组的额外切换主要来自后台 `cat trace_pipe` 进程，两者的量级都与 baseline 同档，说明 ftrace 内核 ring buffer 插桩本身对调度路径几乎无干扰。

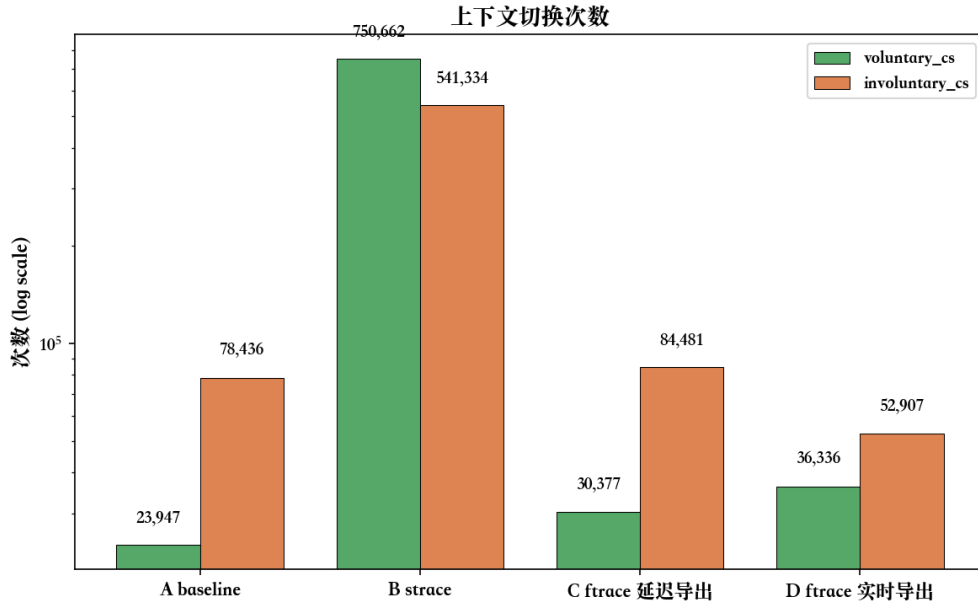


图 7: 四组的自愿与非自愿上下文切换次数（对数坐标）

阶段占比漂移 图 8 显示了四组各阶段占比相对 A 组 baseline 的偏移（pp = percentage point）。B strace 组的 `obs_pct` 漂移高达 +34.8 pp，`wait_pct` 反向漂移 -31.6 pp，意味着 strace 不仅整体拉慢主循环，还改变了阶段时间结构：摄像头链路中的系统调用被 `ptrace` 反复拦截，导致观测阶段被严重拉长，`wait` 阶段反而被挤压；这种结构性扭曲使得 strace 下采集的阶段占比已经无法代表真实负载。C 与 D 两组的所有阶段漂移都在 ± 8 pp 内，其中 `wait/action` 漂移均 ≤ 1 pp，`obs_pct` 与 `inference_pct` 在 ± 7 pp 之间小幅互相借调，没有出现 B 组那种数量级的结构变化；因此可以判断 ftrace 两种方案对阶段时间结构的影响可控。

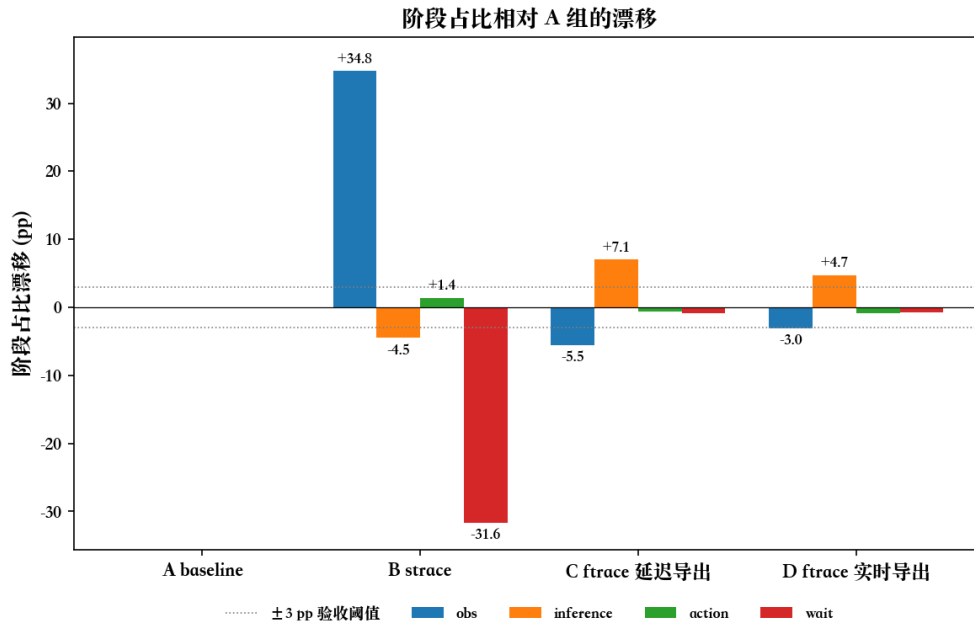


图 8: 四组的四阶段占比相对 A 组 baseline 的漂移 (pp), 灰色虚线为设计阶段提出的 ± 3 pp 验收阈值

本文工具链选型 正式实验的内核态时序数据采用 **ftrace 延迟导出** 方案：其上下文切换次数与阶段时间结构均与 baseline 同档，是单 episode 扰动最小的方案。仅在 episode 时长延长导致 ring buffer（本文配置每 CPU 32 MB，4 核共 128 MB）接近溢出时，才改用 ftrace 实时导出作为后备；其代价是 trace 数据量约为延迟导出的 4.9 倍（823 MB vs 168 MB），但量级仍可承受。strace 不参与最终定量分析，仅用于第三章早期对系统调用类型分布的定性探索。

内核源码级插桩 除上述标准 syscall tracepoint 外，本文还在内核源码中通过 `trace_printk` 插入了自定义计时桩，将更精细的时间拆分信息直接写入 ftrace ring buffer，与标准 tracepoint 事件混合输出。标准 tracepoint 只记录系统调用入口参数和出口返回值，无法拆分调用内部的阻塞睡眠时间与 CPU 处理时间，也无法将文件描述符解析为设备名。

本文在两处关键路径上实现了自定义插桩：

1. **pselect6** (`fs/select.c`, `do_select()` 函数): 在函数入口通过 `ktime_get()` 记录起始时间，在 `poll_schedule_timeout()` 前后打点计算阻塞睡眠时间，函数退出时输出总耗时、睡眠时间、上下文处理时间、返回值、就绪 fd 编号及通过 `d_path()` 解析的设备名。例如，trace 中可直接观察到 “`fd=8, file=video0`” 表

示手眼摄像头帧就绪，“fd=9, file=video2”表示固定摄像头帧就绪，每次等待约 30 ms，睡眠时间约占总耗时的 99.7%。

2. `futex(kernel/futex/syscalls.c, do_futex())` 函数: 类似地在入口和 `futex_wait_multiple`

前后打点，输出操作码 (op)、返回值、总耗时、CPU 时间和睡眠时间。该拆分可以直接区分 `futex` 快速解锁 (`sleep≈0`) 与长时间等待 (`sleep` 接近 `total`)，从而判断 Python 线程间的锁竞争程度。实测数据显示 WAKE 操作 (`op=1`) 耗时在微秒级，而 WAIT 操作 (`op=0`) 的睡眠时间最高可达 1.8 s。

上述信息无法通过标准 `tracepoint` 或 `kprobe` 动态插桩获得：标准 `tracepoint` 不提供内部时间拆分；`kprobe` 回调中调用 `d_path()` 解析 `fd` 路径存在死锁风险，而 `do_select()` 原生上下文中调用是安全的。代价是需使用自编译内核，对本文固定的实验平台不构成额外负担。

[来源：实验/04_ 实验环境与测量工具校准/结论.md，内核数据搜集分析.md]

3.2 感知阶段性能剖析

感知阶段对应主循环中的 `robot.get_observation()`。在本文的 SO-101 实验配置中，一次 `observation` 由两类数据组成：两路 OpenCV 摄像头图像，以及一个 follower 机械臂的 6 个舵机当前位置。其中两路摄像头分别为 `handeye` 和 `fixed`，分辨率均为 640×360 ；舵机状态来自 6 个 STS3215 舵机的 `Present_Position` 寄存器。

从代码结构看，SO-101 follower 的观测函数先同步读取 6 个舵机位置，随后遍历两路相机，取回后台线程已经预取的图像。这意味着 `obs` 阶段并不是单一 I/O，而是“舵机同步读 + 两路相机异步取帧”的组合。舵机读取与执行阶段共用同一条 `/dev/ttyACM0` 半双工总线，协议和内核路径高度重合，因此本节只点明其在 `obs` 中的位置，详细分析放在 §3.4。

3.2.1 摄像头异步读取机制

LeRobot 的 OpenCV 相机并不是在主线程里每次直接阻塞读取硬件帧。每个相机对象内部维护一个后台读线程：只要没有收到停止信号，它就循环调用 OpenCV 读取摄像头，把最新图像写入共享缓存，并设置帧就绪事件。主线程中的 `async_read()` 不再直接面对摄像头设备，而是等待这个事件；事件到达后，它只需短暂获取锁，从共享缓存取走最近一帧，然后清除事件标志，等待下一次更新。这里不是两个线程互通知：后台线程负责生产帧并设置事件，主线程负责等待事件并清除事件。

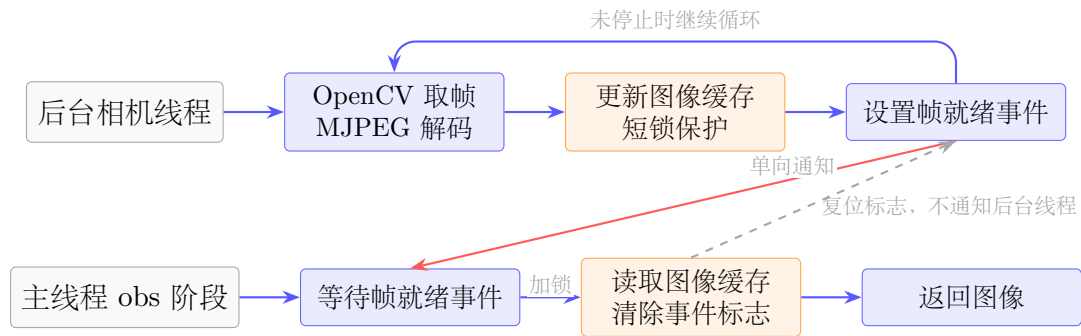


图 9: OpenCV 摄像头异步读取中的后台循环、单向事件通知与主线程取缓存关系

后台线程会调用 OpenCV 的读帧接口，从 V4L2 摄像头设备取出一帧，并在用户态完成 MJPEG 解码和颜色格式整理。整理后的 RGB 图像被写入 `latest_frame`，这个共享缓存由 `frame_lock` 保护。`new_frame_event` 只承担“帧已就绪”的单向通知作用；主线程取走缓存帧后会复位事件标志，避免下一轮直接读到旧帧，但这个复位动作不会反向唤醒或驱动后台相机线程。

因此，`async_read()` 的“异步”含义是：摄像头硬件读取和 MJPEG 解码提前在后台线程中进行，主线程在 `obs` 阶段只等待事件并读取最近一帧。如果后台线程已经准备好图像，主线程很快返回；如果主线程跑得比相机帧率更快，则会在等待新帧事件时阻塞。

3.2.2 是否进入内核

摄像头路径中有三类等待，需要区分其含义。

位置	是否可能进入内核	含义
主线程等新帧事件	可能进入内核	线程同步等待。对应 <code>new_frame_event.wait()</code> ，主线程等后台相机线程把新帧准备好
主线程取缓存帧	通常不进入内核	短锁保护。对应 <code>frame_lock</code> ，只是在读写 <code>latest_frame</code> 时避免两个线程同时改同一块缓存
OpenCV 后台线程取帧	会进入内核	设备可读等待。后台线程通过 <code>select()</code> 等待 V4L2 摄像头有帧可取，随后再执行出队操作取得该帧

也就是说，主线程的 `async_read()` 主要是在等 Python 线程同步事件；真正深入 V4L2、USB 摄像头驱动和 `videobuf2` 队列的是后台读线程。这也是为什么主线程

观测耗时可能只有毫秒级，而摄像头后台线程的 `pselect6` 等待会接近 30 fps 的帧间隔。

3.2.3 OpenCV 到内核的边界

论文正文不展开完整源码栈，只保留性能分析需要的边界关系：后台相机线程首先进入 OpenCV 的 `VideoCapture`，随后通过 `glibc` 发起 V4L2 相关系统调用，最终进入 Linux 的 V4L2、UVC 和 `videobuf2` 队列。

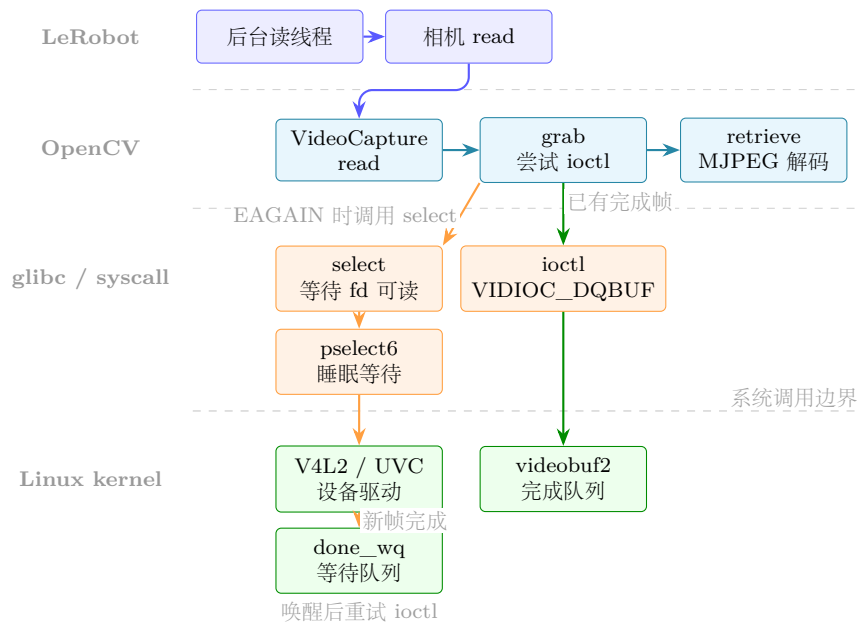


图 10: OpenCV 摄像头后台线程从用户态到 V4L2 内核队列的关键路径

图 10 中有两条需要区分的路径：等待设备可读通常发生在 `select()` / `pselect6` 一类调用上；真正取出已完成图像缓冲区时，OpenCV 使用 `VIDIOC_DQBUF` 将一个已经由驱动填好的 V4L2 buffer 从完成队列中出队。

需要注意的是，`VIDIOC_DQBUF` 并不是把整帧像素从内核重新拷贝一遍。在当前 OpenCV V4L2 路径中，摄像头 buffer 通过 `mmap` 暴露给用户态；出队操作主要返回 buffer 的编号、有效字节数和时间戳等元数据。MJPEG 到 BGR 图像的解码仍然发生在用户态 OpenCV/libjpeg 中，这部分才是 ARM CPU 上摄像头链路中更明显的计算开销。

完整的 OpenCV、glibc 和 Linux 内核源码级 CallStack 与参数说明，不放在论文正文中展开，单独记录在 `robotdoc/摄像头相关/OpenCV 到 glibc 到内核 CallStack 源码解析.md`。

3.2.4 时间剖析

在 30 fps 摄像头配置下，单路摄像头物理帧间隔约为 33.3 ms。实测中，后台线程在 `pselect6` 或 `V4L2` 等待路径中阻塞约 23–32 ms，符合摄像头帧率上限；MJPEG 解码仍在用户态 `OpenCV/libjpeg` 中完成，在 ARM CPU 上属于摄像头链路的主要计算开销。由于 LeRobot 采用后台预取，主线程 `obs` 阶段看到的是“取缓存帧”的时间，而不是完整的摄像头端到端采集时间。因此，主线程通常不会被摄像头硬件读取、`V4L2` 等待或 MJPEG 解码直接阻塞；这些耗时主要由后台相机线程承担。主线程只有在请求图像时后台线程尚未准备好新帧，才会短暂等待 `new_frame_event`，随后只是持锁读取最近缓存帧。

实测的逐层耗时分布与分层架构图见实验 06(摄像头感知层级耗时剖析, [robotdoc/实验/06_摄像头感知层级耗时剖析/](#))。该实验通过 `ftrace` 内核插桩和 Python `perf_counter` 双层时间戳，量化了从 `pselect6` 阻塞到 `tensor` 入模的层级耗时，并区分了“端到端时延”与“主线程感知耗时”在异步预取下的差异。

[来源: *OpenCV* 视频流内部实现.md, *OpenCV* 到 *glibc* 到内核 *CallStack* 源码解析.md, *pselect6* 最终结果.md, 实验数据/06_摄像头感知层级耗时剖析/README.md]

3.3 推理阶段性能剖析

本节专门分析 SO-101 在线控制循环中的 ACT 策略推理阶段。这里的“推理阶段”并不只是一次 `ACT.forward()`，而是从 `predict_action()` 收到 `observation` 开始，经过动作缓存判断、必要时的观测预处理、ACT 模型前向、动作队列写入、再到动作反归一化返回的完整路径。

当前实验配置中，ACT 的 `chunk_size` 与 `n_action_steps` 均为 100，单次完整模型推理输出 $(B, \text{chunk_size}, \text{action_dim}) = (1, 100, 6)$ 的动作序列。因此在线循环的大多数帧并不会重新执行模型前向，而是直接从 `_action_queue` 中取出上一次推理缓存的下一帧动作。这一区别是理解第 3.3 节所有时间数据的前提：`chunk_size` 主要是在时间上分摊单次 ACT 前向开销，而不是让一次 ACT 前向本身变快。

3.3.1 select_action 与动作队列

LeRobot 在 `predict_action()` 外层先检查策略对象的动作队列。如果 `_action_queue` 非空，主循环直接 `popleft()` 取出缓存动作，再通过 `postprocessor` 反归一化为真实舵机目标；这一帧会跳过图像 `tensor` 转换、`preprocessor` 和 `ACT.forward()`。只有当队列为空时，系统才会走完整推理路径：把 `numpy observation` 转为 `PyTorch Tensor`，

执行归一化预处理,调用 `policy.select_action()`,继而触发 `predict_action_chunk()` 和 `ACT.forward()`。模型一次产生 100 帧归一化动作,写入队列后立即弹出第 0 帧返回;后续 99 帧由队列快速路径逐帧取用。

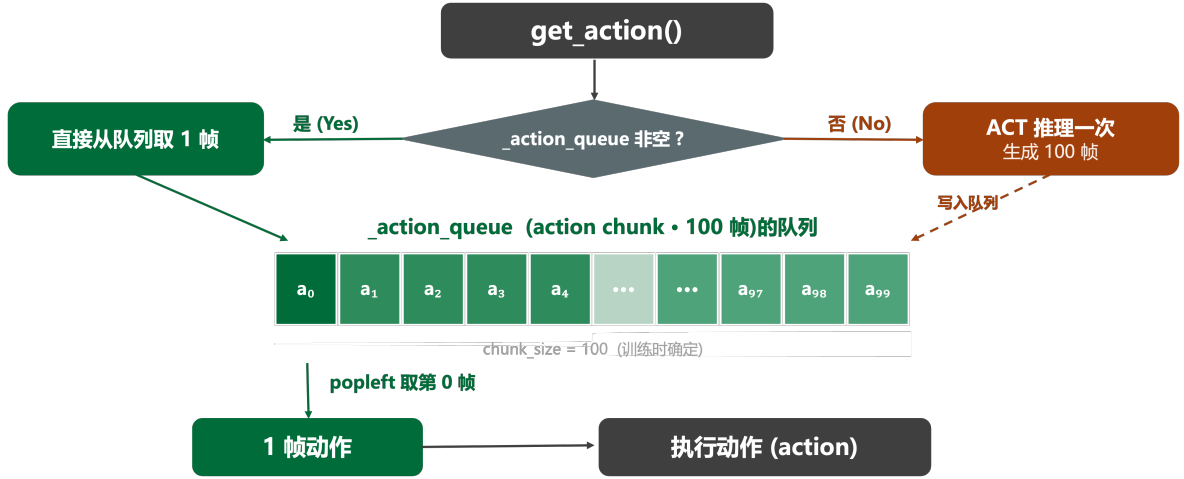


图 11: ACT 在线推理中动作队列快速路径与完整推理路径

图 11 说明了 `chunk_size=100` 下的两种时间尺度: 队列非空帧只承担动作取队列和反归一化, 属于轻量路径; 队列为空帧才承担完整 ACT 前向, 属于重推理路径。因此, 按 episode 统计得到的 inference 占比, 本质上是少数重推理帧在 100 帧执行周期内被均摊后的结果。

3.3.2 一次完整 ACT 推理的数据流

队列为空时, `predict_action()` 会先把原始 observation 整理成 ACT 可接受的 batch。两路相机图像从 (H, W, C) 的 `uint8` numpy 数组转换为 $(1, 3, 360, 640)$ 的 `float32` Tensor; 关节状态从 $(6,)$ 转为 $(1, 6)$ 。随后 `preprocessor` 使用随 checkpoint 保存的训练集 mean/std 对图像、state 进行归一化。进入 `ACT.forward()` 后, 推理态下不运行训练用 VAE encoder, 而是使用全零 latent。

图 12 中, 视觉分支是主要计算来源。每路 640×360 图像经 ResNet18 的 `layer4` 得到 $512 \times 12 \times 20$ 特征图, 两路相机共形成 480 个视觉 token; 再加上 1 个 state token 和 1 个全零 latent token, 构成长度 482、维度 512 的 Transformer Encoder 输入序列。Decoder 使用 100 个可学习 query, 最终通过线性层输出 100 帧、每帧 6 维的归一化动作。



图 12: 队列为空时一次完整 ACT 推理的数据流与模型结构

3.3.3 时间剖析：chunk_size 参数扫描

实验 01 在树莓派 5 上对同一 ACT 抓取瓶子策略扫描 `chunk_size`，取值覆盖 $\{1, 3, 5, 10, 20, 50, 100\}$ ，每个取值固定记录 20 个 episode，其它参数（模型权重、分辨率、目标 fps）保持一致。原始数据与绘图脚本保存在 [来源：实验 01: `chunk_size` 参数扫描]。后文先看四个阶段的时间占比如何随 `chunk_size` 变化，再单独看 FPS 的增长曲线，最后用代表性数值表汇总。

阶段时间占比随 `chunk_size` 的演化 图 13 给出了 obs/inference/action/wait 四阶段平均占比（20 episode 均值）随 `chunk_size` 的变化曲线。`inference_pct` 从 $cs=1$ 时的 97.6% 单调下降到 $cs=100$ 时的 36.5%，对应 `wait_pct` 从 0% 单调上升到 43.3%；`obs_pct` 从近 0% 缓慢增长到 15.4%，`action_pct` 始终保持在 5% 以内。这条主曲线说明 `chunk_size` 越大，单帧分摊到的推理工作量就越小，控制循环把更多预算还给 `wait`（即在帧内等待 fps 节拍），推理不再压满整个 33.3 ms 时间预算。观测和执行阶段的占比上升只是相对结果——其绝对耗时基本恒定，只是因为 inference 缩短才在比例上变得显眼。

FPS 随 `chunk_size` 的非线性增益 图 14 给出了实测平均 FPS 随 `chunk_size` 的变化曲线。FPS 从 $cs=1$ 的 0.8 上升到 $cs=100$ 的 18.7，但增长曲线高度非线性： $cs=1 \rightarrow 10$ 时 FPS 提升约 6.6 倍 (0.8 $\rightarrow$ 5.3)， $cs=10 \rightarrow 50$ 时再提升约 2.9 倍，而 $cs=50 \rightarrow 100$ 时增益仅约 1.2 倍 (15.4 $\rightarrow$ 18.7)，呈现典型的边际递减。原因是单次完整 ACT 推理的

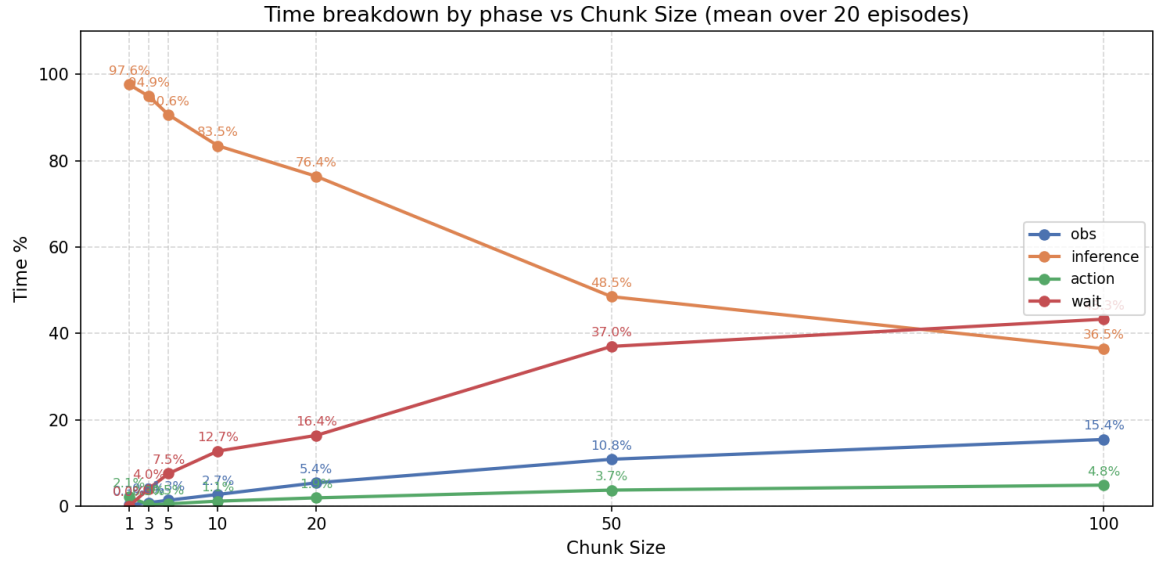


图 13: chunk_size 对控制循环阶段时间占比的影响

绝对耗时接近恒定 ($\approx 1-1.5$ s/次)，分块机制只是把这一次重推理在时间上摊薄到更多控制帧上，而不是降低 `ACT.forward()` 本身的计算量；当 `cs` 增大到推理被摊薄到接近 `fps` 目标时，等待阶段就成为主循环新的瓶颈，进一步增大 `cs` 不再带来等比增益。

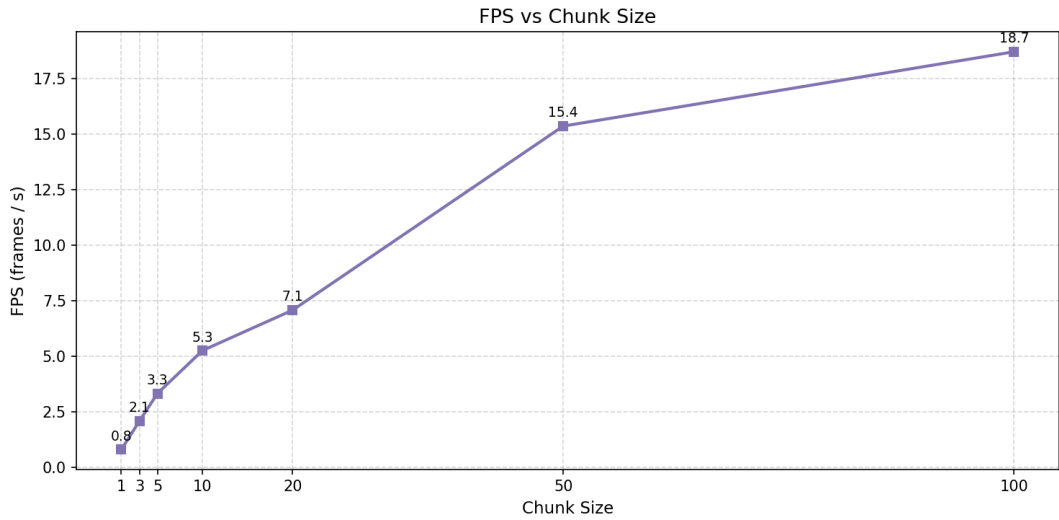


图 14: chunk_size 对控制循环 FPS 的影响

`cs=100` 在树莓派 5 上能跑到 ≈ 18.7 fps，距 30 fps 目标仍有约 38% 缺口；而此时 `wait_pct` 已升至 43.3%，说明继续增大 `cs` 已经不能进一步把 `fps` 推到 30——要逼近目标必须从减小单次推理绝对耗时入手，也就是后文（第四章）讨论的优化方向。

3.3.4 单次 ACT 推理耗时分解

上面的 episode 级统计能够说明 `chunk_size` 如何分摊推理开销，但还不能回答“一次完整 ACT 推理内部究竟慢在哪里”。实验在 `modeling_act.py` 的 `select_action()` 和 `ACT.forward()` 内插入计时点，通过环境变量 `LEROBOT_PROFILING=1` 启用，数据直接写入 CSV。在树莓派 5 上记录了 6 次队列为空的完整推理，结果汇总于表 2。

模块	平均耗时 ms	标准差 ms	占完整推理比例
<code>select_action</code> 完整路径	1924.8	414.9	100.0%
ResNet18 + 投影	1281.1	259.9	66.6%
Transformer Encoder	569.5	506.3	29.6%
Transformer Decoder	44.0	4.4	2.3%
<code>action_head</code>	0.7	0.8	<0.1%
队列非空快速路径（对照）	<1 ms	—	—

表 2: ACT 单次完整推理（队列为空时）的逐模块耗时分解

图 15 给出了各模块的耗时柱状图。

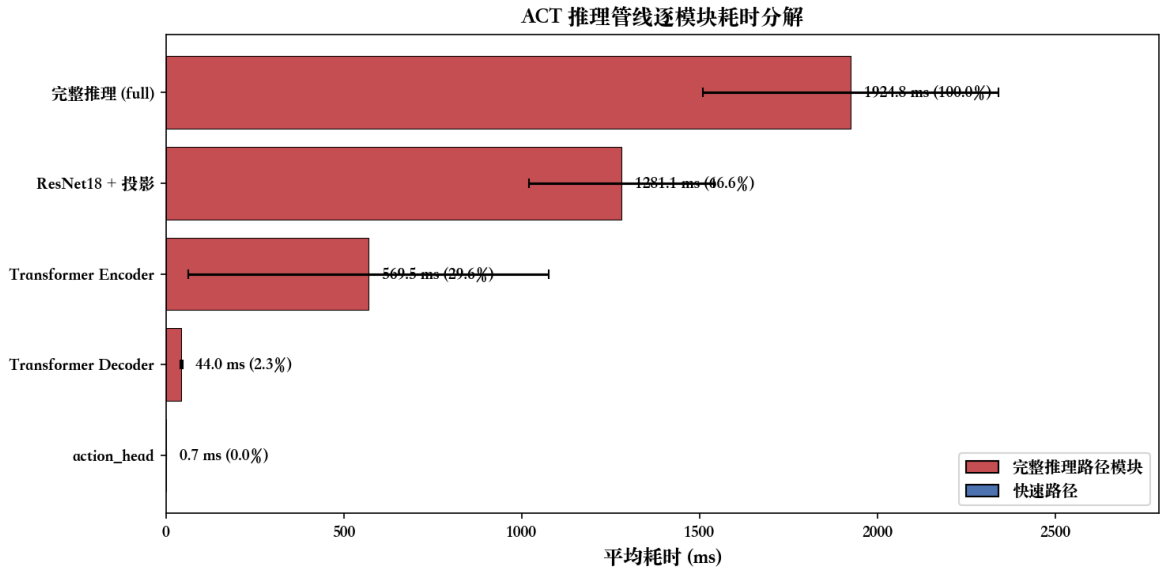


图 15: ACT 推理管线逐模块耗时分解

关键发现：ResNet18 视觉特征提取（两路 ResNet18 backbone + 位置编码 + 1×1 投影）占据完整推理时间的 **66.6%**（平均 1281 ms），是第一瓶颈。Transformer Encoder 次之，占 29.6%（569 ms）。Decoder 和 `action_head` 占比极低（2.3% 和 <0.1%），不是优化重点。队列非空快速路径耗时不足 1 ms，与后处理相当，验证了动作缓存机制的有效性。

3.3.5 PyTorch 线程行为与推理优化研究现状

PyTorch 在 ARM CPU 上使用 OpenMP 并行框架，通过 ATen 线程池调度矩阵运算。系统调用追踪显示 futex 调用共计 64,569 次 / 40 s episode，其中 27 线程并行累计等待时间达 3,252 s，说明多线程同步开销在 ARM CPU 上不可忽略。

推理优化方向（如 INT8 量化、算子融合、线程亲和性、NEON SIMD 加速等）已有大量文献研究，此处不再展开。

3.4 执行阶段性能剖析

执行阶段的核心是机械臂六个 STS-3215 舵机的位置指令下发（`sync_write`）。为完整描述舵机链路，本节同时纳入感知阶段中物理通道完全重合的舵机位置读取（`sync_read`）。两者共用同一条 `/dev/ttyACM0` 半双工 TTY 总线、相同的 SCServo 协议帧结构与 USB 串口内核路径，仅在数据包内的指令字节（0x82 vs 0x83）和方向（写 vs 读 + 回包）上不同。

3.4.1 LeRobot 执行路径软件栈

图 16 展示了执行阶段从 `record.py` 触发 `send_action()` 到 `scservo_sdk` 的关键类关系与调用关系。

`get_observation()` 和 `send_action()` 分别对应读取舵机当前位置（`sync_read`）和写入目标位置（`sync_write`）的两条路径，共享 MotorsBus 层（归一化 / 符号编解码）和 Feetech 层（协议组包）。关键数据点：Present_Position 寄存器地址 56，写 2 字节；Goal_Position 寄存器地址 42，写 2 字节。读路径需等待 6 个舵机依次回包（半双工总线），写路径发完即返回，无回包。

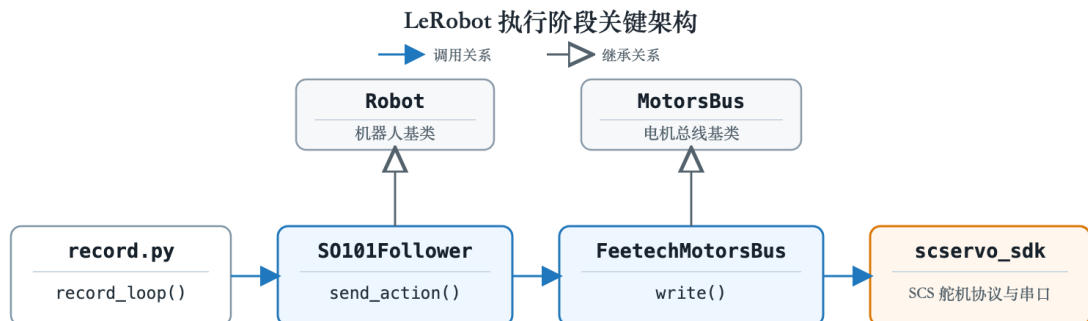


图 16: LeRobot 执行阶段关键类关系与调用关系

3.4.2 sync_write 写链路：从电机总线到串口写入

图 17 展示了 `send_action()` 触发的完整写链路，横跨五个软件层：MotorsBus（归一化/编码）→ GroupSyncWrite（组包）→ GroupSyncWrite.syncWriteTxOnly()（协议帧）→ PortHandler（串口管理）→ `ser.write()`（内核写入）。

`_setup_sync_writer` 设置寄存器地址（`start_address=42`, `Goal_Position`）与数据长度（2 字节），`addParam(id, data)` 将每个电机的目标位置序列化为 `[lo, hi]` 存入 `data_dict[id]`；随后 `makeParam()` 将 6 个电机数据拼成 16 字节负载，`syncWriteTxOnly()` 追加协议头与校验字节，构造出完整的 26 字节 SYNC_WRITE 帧（FF FF FE 16 83 2A 02 `[id lo hi] × 6` CS），广播 ID `0xFE` 意味着无需等待任何舵机回包；最终 `port.writePort(pkt)` 在 `is_using=False` 的条件下调用 `ser.write()` 将帧写入串口，写完即返回，无 RX 阶段，随后 `clearParam()` 重置组包缓冲区。

写链路的核心特性是“发完即返回”——广播帧不要求舵机回包，Python 侧耗时约 1–3 ms，瓶颈为 USB → UART 的物理传输，而非软件开销。

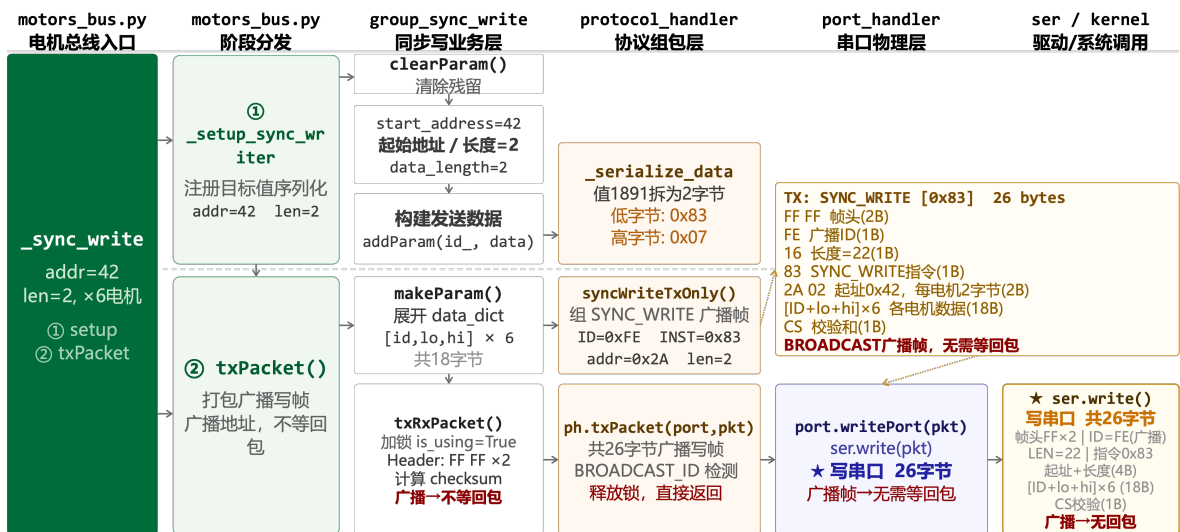


图 17: sync_write 写链路: `motors_bus.py` → `GroupSyncWrite` → `ProtocolHandler` → `PortHandler` → `ser.write()`

3.4.3 sync_read 读链路：从请求帧到逐电机轮询回包

图 18 展示了 `get_observation()` 触发的读链路。与写链路共享前三层软件栈，但在 `PortHandler` 层之后分叉为完整的“发送请求帧 + 逐电机轮询”双阶段。

关键数据路径如下：

1. 请求帧发送: `syncReadTx()` 构造 SYNC_READ 帧（指令字节 `0x82`，寄存器地

址 0x38/56, 读长度 2), 经 `ph.txPacket()` 调用 `writePort()` → `ser.write()` 写出。帧格式: FF FF FE 0A 82 38 02 01 02 03 04 05 06 CS。

2. 逐电机回包: `rxPacket()` 对 `data_dict` 中每个 ID 依次调用 `readRx(port, id, 2)`, 后者调用 `ph.rxPacket(port)` → `ser.read(n)`。`ser.read(n)` 内部以 while 轮询加超时退出, 等待每个电机按 ID 顺序在半双工总线上依次发回状态帧 (FF FF id 04 00 lo hi CS)。
3. 数据解析: `getData(id, 56, 2)` 计算偏移量 `offset = 56-56 = 0`, 取 `data[0]`, `data[1]` 合成 16-bit 位置值 (SCS_MAKEWORD), 最终返回 {1: 2381, ..., 6: 1500} 形式的字典。

读链路耗时约 1.2 ms, 主要来源于 6 次串行回包等待。半双工总线在等待回包期间不能发送任何其他指令, 这是写链路无法与读链路并行化的物理根因。

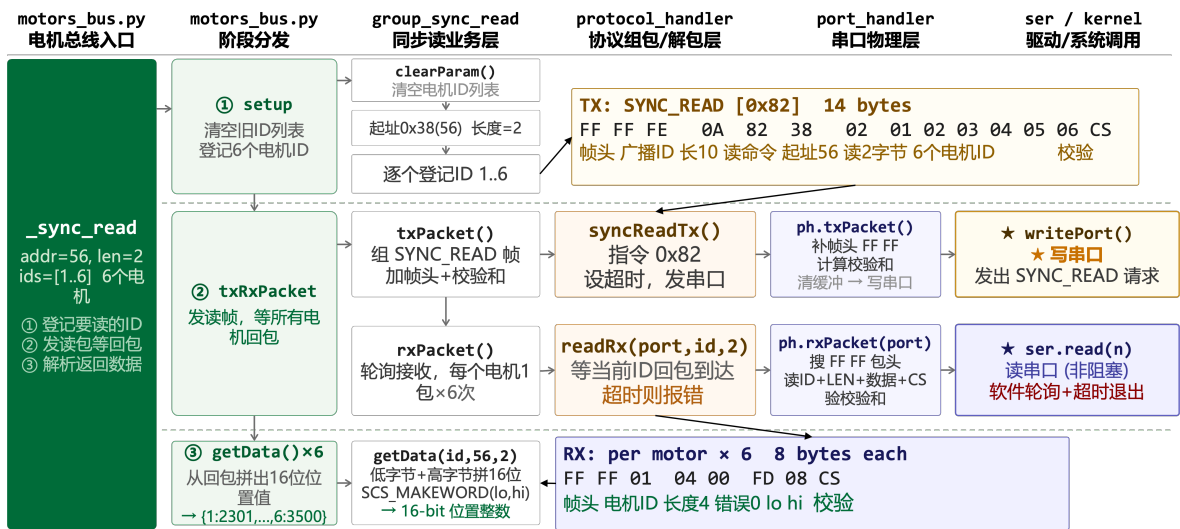


图 18: `sync_read` 读链路: `motors_bus.py` → `GroupSyncRead` → `ProtocolHandler` → `PortHandler` → `ser.read()` 轮询

关键性能结论: 综合 `sync_read` 与 `sync_write` 两条路径可以看出, 执行阶段的主要耗时并不来自 Linux 系统调用本身。`write` 系统调用仅约 38 μ s, `read` 也处于类似量级; 实际观测到的 `sync_read` 约 1.2 ms、`sync_write` 约 1–3 ms, 主要由 1 Mbps 波特率下的 SCServo 总线传输决定。具体来说, Sync Read 指令帧约 14–15 字节, Sync Write 指令帧约 26 字节 (6 ID × 4 字节位置数据, 加上包头与校验), 串口物理传输时间构成耗时主体。因此, 执行阶段瓶颈位于半双工舵机总线和协议物理层, 而不是 Python、SDK 封装或系统调用路径。进一步地, 同一时刻总线只能容纳一个方向的传输, 若将读写简单异步并发, 反而可能造成总线竞争和指令帧损坏;

同时，`send_action()` 中 read-modify-write 的安全限幅链路依赖同步读返回的实时位置，异步化会破坏这一控制不变量。因此，舵机读写不适合作为本文主要异步优化方向（详见 §4.1）。

第四章 优化

4.1 异步化路径选择

本章优化的直接目标是提高在线推理过程的有效 FPS。第三章已经表明，在树莓派 5 上，单纯依靠同步流程难以达到 30 FPS；因此需要从控制循环结构入手，寻找可以从主路径中移出的阻塞环节。相机读取已经采用后台线程异步获取最新帧，主循环只取最近一次结果。这一机制说明，在不改变任务语义的前提下，部分 I/O 或计算过程可以提前执行，从而减少主循环等待时间。

基于这一思路，本文首先考察感知、执行和推理三个阶段中可异步化的事件。舵机读写看似也可以交给后台任务，但 SO-101 机械臂的多个舵机共享同一条 TTL 半双工总线，同一时刻只能进行一次读或写。为了避免后台写入和主线程读状态同时占用总线，必须在总线访问外层加互斥锁；一旦加锁，读写仍然只能按顺序执行，异步线程只是把原来的串行访问换成“持锁串行 + 线程切换”，并不能真正隐藏总线时间。舵机写入还涉及动作限幅的连续性，延迟到后台执行后，错误反馈和安全判断都会变得滞后。

舵机读取同样不能直接照搬相机的后台线程模式。若增加一个后台读线程，它仍然要访问同一条舵机总线，并会与主线程写目标位置竞争同一把总线锁；锁住之后，读写事务仍按顺序通过总线，异步读无法形成真正并行。一种较温和的方案是采用流水线读：本帧接收上一帧请求的状态，同时发出下一帧请求。该方案可以避免后台线程和总线锁竞争，但代价是观测天然滞后一帧，并且仍需改动底层舵机通信封装。第三章测得单次舵机读取约 1.2 ms，相对 ACT 推理约 2 s 的主瓶颈而言收益过小，因此不作为本章主要实验方向。

相比之下，推理阶段既是主要耗时来源，又天然具备与动作执行重叠的时间窗口。在 `chunk_size=100`、目标 30 FPS 的配置下，一个 action chunk 执行约 3.3 s；同步模式在这段时间内并不会提前计算下一段动作。因此，将下一次模型推理移出主控制路径，使其在当前 chunk 执行期间并行进行，是更符合当前瓶颈结构的异步化方向。后续 4.2 节分析 LeRobot 的双进程推理机制，4.3 节进一步通过 hold 参数扫

描述验证该方案的实际收益和边界。

4.2 推理异步化

4.2.1 设计思路

回到 4.1 节提出的问题：在同步流水线下，每完成一次模型推理后，系统还要顺序执行整段动作块，这意味着推理时间和动作执行时间被直接相加，有效 FPS 只能由两者的总耗时决定，在本平台上约为 19 FPS。如果能把下一次推理重叠到当前动作块的执行阶段内，就有机会把系统吞吐恢复到更接近目标控制频率的水平。这就是推理异步化的核心设计动机。

与舵机读写不同的是：推理过程纯粹是 Python+PyTorch 计算，不与任何半双工硬件资源耦合，理论上具备真正并行的物理基础；LeRobot 上游也已为此提供了完整的双进程实现，无需自行从零开发。因此本节的工作不再是“评估能不能做”，而是“评估在树莓派 5 + 当前 ACT 模型的具体参数下，做了能不能带来实际收益”。

4.2.2 实现结构

如图 19 所示，LeRobot 将控制与推理拆分为两个独立进程，通过 gRPC（远程过程调用框架）交换观测与动作序列。

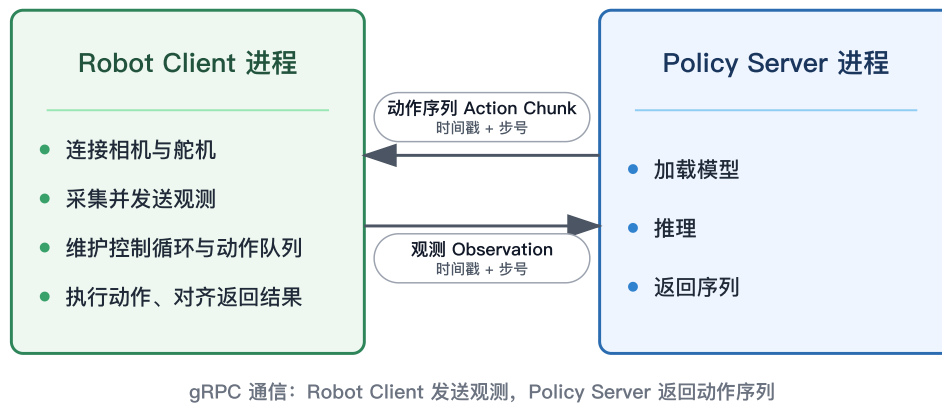


图 19: 异步推理架构：Robot Client 与 Policy Server 是两个独立进程。前者负责硬件连接、观测采集、控制循环与动作执行，后者负责加载模型、完成推理并返回动作序列；二者通过 gRPC 交换带时间戳和步号的观测与动作序列。

其中，`robot_client.py` 进程负责连接相机和舵机、维持控制循环，并将当前观测发送给 `policy_server.py` 进程；`policy_server.py` 进程负责加载 ACT 模型，在收到观测后完成一次动作序列预测，再将结果返回给 `robot_client.py` 进程。这样，

动作执行仍由本地控制循环逐帧推进，而下一段动作的生成可以在另一进程中提前进行。

跨进程传输的数据对象包含时间戳和步号两个关键信息。前者用于衡量观测发送、推理计算和动作返回之间的总时延，后者用于判断一段动作序列对应的是哪一时刻的控制状态，从而在异步回包到达时完成帧级对齐。

在 `robot_client.py` 进程内部，主线程持续执行控制循环，后台接收线程则专门等待 `policy_server.py` 进程返回结果。二者通过一个带互斥保护的动作队列共享数据。控制循环一方面从队列头部取出当前应执行的动作，另一方面持续监测队列剩余长度，以决定何时触发下一次推理请求。与此同时，系统还维护最近一次已执行动作的步号、当前动作块长度以及队列是否即将耗尽等状态，用于支持后续的触发、对齐和兜底逻辑。

基于上述结构，异步推理的运行机制可以归纳为三个动作队列上的规则。**第一是提前触发**：`robot_client.py` 进程不会等当前动作块完全耗尽后再请求下一次推理，而是在剩余动作下降到一定比例时提前发出新观测。LeRobot 默认将这一比例设为 0.5，即动作块尚余一半时就启动下一次推理。这样做的目的，是让新动作块尽可能在旧动作块执行完之前返回，从而把推理时间隐藏在执行阶段内部。

第二是时序对齐与融合：新动作块返回后，`robot_client.py` 进程不会简单覆盖旧队列，而是依据步号与当前执行位置进行对齐。已经过时的动作会被直接丢弃，尚未执行但与旧动作块覆盖到同一未来时刻的部分则进行融合，只把最终有效的未来动作重新写入队列。这样可以缓解动作块切换时的突变，也是 temporal ensemble 在异步控制场景中的具体落地方式。

第三是空队列兜底：若推理速度不足，导致动作队列接近耗尽，`robot_client.py` 进程会附带发送一个强制触发标记，要求 `policy_server.py` 进程立即处理当前观测，而不再等待常规去重或合并逻辑。这条路径本质上是异常兜底，用来避免队列彻底清空后控制循环停顿。因此，强制触发出现得越频繁，越说明当前异步流水线离稳定工作区越远，它也就成为后续实验中判断系统健康度的重要观测量。

4.2.3 可行性不等式与 hold 参数

异步推理的稳态行为受四个参数控制：`chunk_size` N 、单次推理耗时 T_{infer} 、目标执行频率 F 、以及提前触发阈值 h （即 `chunk_size_threshold`）。在 LeRobot 当前实现中，触发条件为 `action_queue.qsize() / action_chunk_size <= h`，因此 h 越大，触发越早； h 越小，触发越晚。

令一次推理期间主线程按目标频率会消耗的动作帧数为

$$D = T_{\text{infer}} F. \quad (1)$$

严格来说，调度条件中应使用从观测被 server 取走到动作 chunk 并入 client 队列的端到端返回时延；下文用 server 端记录的 T_{infer} 近似这一时延，网络、序列化和队列调度开销则交由 4.3 节的实测结果反映。为简化推导，以下也忽略 `timestep <= latest_action` 带来的 1 帧端点差异。

代码中存在两个不同层次的约束。第一，触发必须足够早：当队列剩余 hN 帧时发出观测，如果 $D > hN$ ，新动作在当前队列耗尽前无法返回，控制循环必然出现 stall。因此有

$$D \leq hN \iff h \geq \frac{T_{\text{infer}} F}{N}. \quad (2)$$

这只是“本次请求来得及赶上当前队列”的条件。

第二，新 chunk 到货后并不会完整进入队列。`policy_server.py` 返回的动作步号从观测步号开始递增，而 `robot_client.py` 会丢弃所有 `timestep <= latest_action` 的动作。若返回前没有 stall，推理期间已经执行了约 D 帧，因此新 chunk 中真正能进入未来队列的帧数约为

$$N_{\text{eff}} \approx N - D = N - T_{\text{infer}} F. \quad (3)$$

由于同一 server 一次只能处理一个推理请求，下一轮推理仍要消耗约 D 帧；所以稳态不 stall 还需要

$$N_{\text{eff}} \geq D \iff N \geq 2T_{\text{infer}} F. \quad (4)$$

综合两者，异步流水线稳定无 stall 的近似条件应写为

$$\boxed{T_{\text{infer}} F \leq \min(hN, N/2)}. \quad (5)$$

当 $h = 0.5$ 时，两个约束同时退化为 $N \geq 2T_{\text{infer}} F$ 。当 $h < 0.5$ 时，主要风险是触发太晚；当 $h > 0.5$ 时，更早触发可以增加当前队列的等待余量，但不能突破“一个 chunk 至少覆盖两个推理窗口”这一由 $N/2$ 决定的稳态上限。因此在树莓派 5 这种推理慢、执行帧率固定的平台上， h 仍需作为调度参数单独扫描，但它不能替代对 T_{infer} 或 N 的优化。

为直观展示该不等式的物理含义，图 20-22 给出 $N = 100$ 、 $F = 30$ 时三种代表性 T_{infer} 取值下的稳态流水线时间线。图中横轴为帧号（@30 FPS），纵向依次展示执行进程、推理进程与 chunk 队列内容；蓝色块为正常执行帧，黄色块为 Temporal Ensemble 重叠区，红色块为 stall 停顿。

情况 A 对应 $T_{\text{infer}} = 30$ 帧（1 s），此时 $D = 30 \leq \min(hN, N/2) = 50$ ，不等式严格满足。推理进程在旧 chunk 执行至第 50 帧时启动，于第 80 帧前已完成，形成约 20 帧宽的 Temporal Ensemble 重叠窗口，新旧 chunk 对这批帧均有预测，融合后入队，执行全程连续无中断，稳态可达 30 FPS。

情况 B 对应 $T_{\text{infer}} = 50$ 帧（1.67 s），此时 $D = 50 = \min(hN, N/2)$ ，恰好处于临界点。新 chunk 在旧 chunk 最后一帧执行完毕时精确到达，既无 Temporal Ensemble 余量，也无 stall；但对参数扰动几乎没有容错空间，推理轻微抖动即触发 stall。

情况 C 对应 $T_{\text{infer}} = 70$ 帧（2.33 s），此时 $D = 70 > \min(hN, N/2) = 50$ ，不等式不满足，出现周期性 stall。旧 chunk 耗尽后新 chunk 仍在推理中，每周产生约 20 帧 stall；stall 期间 `latest_action` 冻结在第 99 帧，新 chunk 过期帧为 [50, 99] 共 50 帧（非物理意义上的 70 帧），有效帧 [100, 149] 共 50 帧进入队列，稳态有效 FPS $\approx 50/2.33 = 21.4$ 。

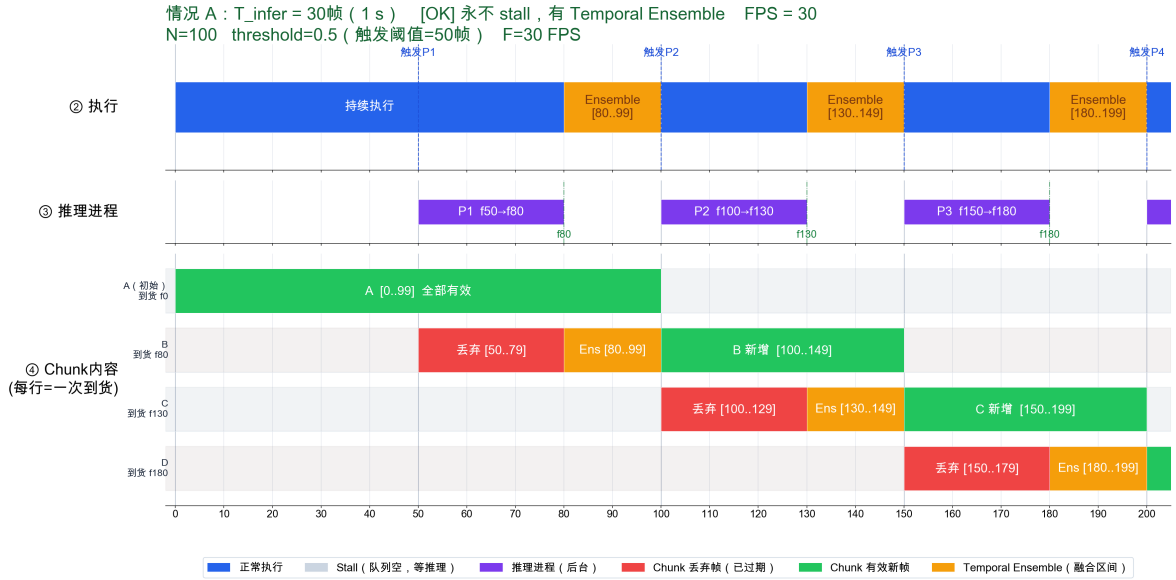
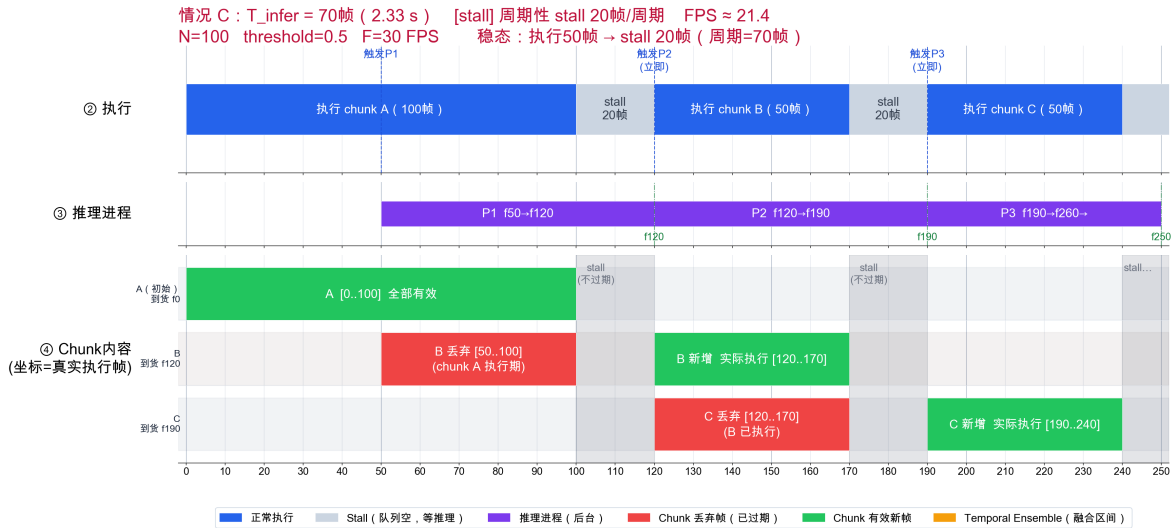
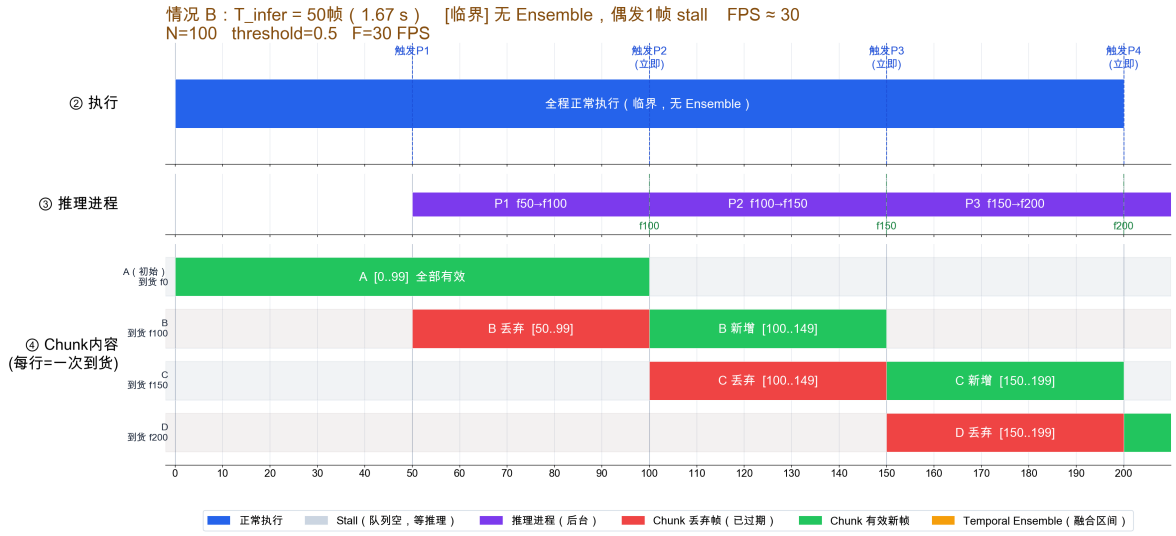


图 20: 情况 A: $T_{\text{infer}} = 30$ 帧，不等式满足，稳态 30 FPS

值得指出的是情况 C 下的一个微妙之处:stall 期间机器人停止执行,latest_action 被冻结，因此当新 chunk 到货时，“过期帧”的判定基准并不是“物理时刻已经走过的帧数” $T_{\text{infer}} \cdot F$ ，而是 stall 之前最后一次执行到的帧号。也就是说，stall 的那段时间“反过来保护”了新 chunk 的有效帧不被丢弃。具体到情况 C，新 chunk 时间戳



[50, 149] 到货时 `latest_action` = 99 (chunk A 最末帧)，有效帧数 = $|[100, 149]| = 50$ 而非 30。这一点必须在公式推导中保留，否则时间线与公式会出现不自洽。

4.2.4 不等式在本平台的代入

代入第三章的实测参数 $T_{\text{infer}} = 2 \text{ s}$ 、 $F = 30 \text{ FPS}$ ：

$$D = T_{\text{infer}}F = 2 \times 30 = 60. \quad (6)$$

在默认 $h = 0.5$ 、 $N = 100$ 时，

$$\min(hN, N/2) = \min(50, 50) = 50 < D, \quad (7)$$

因此异步流水线无法达到完全无 stall 的 30 FPS 稳态。

需要注意的是，不能在 stall 场景下继续使用 $N_{\text{eff}} = N - D$ 来估算有效 FPS。队列耗尽后机器人停止执行，`latest_action` 不再随物理时间前进，所以返回 chunk 的过期部分不会继续扩大。对默认 $h = 0.5$ 而言，进入周期性 stall 后，每轮返回大约仍有 $N/2 = 50$ 帧可执行；理想化有效 FPS 近似为：

$$F_{\text{actual}} \approx \frac{N/2}{T_{\text{infer}}} = \frac{50}{2} = 25 \text{ FPS}, \quad (8)$$

这与后文异步实验中 25 FPS 左右的有效控制频率处在同一量级，也明显高于同步模式的实测约 16–19 FPS。

从 hold 扫描的角度看，更关键的是找到实际临界点：当 hold 小于临界值时，触发太晚，stall 周期性出现；当 hold 越过临界值后，新 chunk 能在旧队列耗尽前到达，理想稳态就应恢复到 30 FPS。因此问题从“异步推理是否可用”进一步细化为：**在动作质量仍可接受的前提下，实际临界 hold 在哪里，以及推理本体还需降到多快才能给该临界点留下足够余量。**4.3 节即围绕该参数展开实验扫描。

需要指出的是，上述不等式是在“每次触发都会立即启动一次新推理”的理想假设下推导的。在 LeRobot 的实际实现中，`policy_server.py` 还会对观测做帧号去重和相似性过滤；同时，`robot_client`、`policy_server`、相机线程和串口通信在树莓派 5 上竞争同一组 CPU 核心。因此，实验中真正起作用的不是纯粹的模型前向耗时，而是从触发观测到新动作 chunk 合并进队列的有效返回时延。下文把这些实现细节统一吸收到 T_{rt} 中，再用 4.3 节的实测数据确定实际临界点。

4.2.5 FPS 与 hold 的定量关系

4.2.3 节的不等式只给出了稳定无 stall 的边界，却没有回答一个更具体的问题：在到达该边界之前，也就是确定会发生 stall 的前段区间，有效 FPS 应该如何随 hold 变化？为避免把代码实现中的抖动误写成理论规律，本节只分析理想调度下的主导关系：**临界点之前 FPS 随 hold 上升，临界点之后饱和到目标帧率 30 FPS**。以下推导只针对临界点之前的 stall 段，即 $h < h_{\text{crit}}$ 且 $T_{\text{rt}} > hN/F$ 。实验中的 $h = 0.60$ 仅作为偏早触发的参考点，不纳入该段公式。

令 T_{rt} 表示从 client 触发观测到新动作 chunk 合并进队列的有效返回时延。注意这里不再把 T_{rt} 强行解释成“实际执行掉的帧数”：一旦发生 stall，机器人在推理期间实际最多只会继续执行触发时剩余的 hN 帧，之后就停住等待。

理想化地看，一个周期里只需要数清三件事：执行了多少帧、计算被隐藏了多少、剩下多少时间必须 stall。触发发生在队列剩余 hN 帧时，因此计算期间最多能被旧队列继续执行的时间隐藏 hN/F 。在本节讨论的前段区间中，返回时延一定更长，所以多出来的部分就是 stall：

$$T_{\text{stall}}(h) = T_{\text{rt}} - \frac{hN}{F}, \quad h < h_{\text{crit}}. \quad (9)$$

在临界点之前，新 chunk 返回时旧队列已经耗尽，触发时剩余的 hN 帧也已经执行完毕。因此新 chunk 前部与旧 chunk 重叠的 hN 帧会被判定为过期，真正能作为未来动作执行的帧数约为

$$E(h) = N(1 - h), \quad (10)$$

对应的正常执行时间为

$$T_{\text{exec}}(h) = \frac{E(h)}{F} = \frac{N(1 - h)}{F}. \quad (11)$$

于是一个理想周期的有效 FPS 直接就是“执行帧数 / （执行时间 + stall 时间）”：

$$F_{\text{eff}}^{\text{ideal}}(h) = \frac{E(h)}{T_{\text{exec}}(h) + T_{\text{stall}}(h)} = \frac{N(1 - h)}{T_{\text{rt}} + \frac{N(1 - 2h)}{F}}. \quad (12)$$

当

$$h_{\text{crit}} = \frac{T_{\text{rt}}F}{N} \quad (13)$$

时， $T_{\text{stall}} = 0$ ，有效 FPS 就达到目标值 F ；继续增大 hold 就离开本节计算的 stall

段。在理想模型中，临界点之后不再有 stall，有效 FPS 饱和为 30 FPS。

因此，FPS-hold 曲线的理论形态很简单：临界点前，hold 增大主要是在缩短 stall，FPS 随之上升；临界点后，stall 被消除，FPS 饱和到 30。4.3 节中的实验扫描只需要确定实际部署下的 h_{crit} ：若某个 hold 之后的稳态阶段几乎不再出现 stall，则说明它已经越过临界点；若 episode 平均 FPS 仍低于 30 FPS，应优先从启动阶段 stall、CPU 资源竞争和调度抖动解释，而不是把它理解为理论曲线在临界点后的必然下降。

4.2.6 树莓派 5 单机部署的 CPU 影响

异步推理在理论分析之外还有一项需要说明的实际因素：树莓派 5 只有 4 个 ARM Cortex-A76 核心，而模型推理、控制循环、串口通信和相机处理都在同一台设备上运行。因此，当 `policy_server` 与 `robot_client` 同机部署时，两者不可避免地会发生 CPU 资源竞争，进而抬高实际推理耗时。

本章实验并未额外采用绑核或线程数限制等隔离手段，因此 4.3 节测得的 T_{infer} 、有效 FPS 和停顿频次，都已经包含了这种单机资源竞争带来的影响。换言之，后文实验结果反映的是当前部署方式下的真实运行表现，而不是理想隔离条件下的上界性能。

4.3 异步推理收益的实验验证

4.3.1 实验目的与论证逻辑

4.2 节给出的分段模型说明，异步推理的理想 FPS-hold 曲线应当具有清晰的临界点：临界点之前，hold 越大，触发越早，stall 越短，有效 FPS 随之上升；越过临界点之后，新动作 chunk 能在旧队列耗尽前返回，系统应饱和到目标 30 FPS。因此，本实验的核心目的不是证明“hold 越大越好”，而是在树莓派 5 单机部署条件下寻找这个实际临界点，并解释为什么实测 episode 平均 FPS 没有完全达到理论上的 30 FPS。

评价逻辑分为两层。第一层是**动作可完成性**：若某个 hold 导致抓取轨迹明显漂移或任务成功率下降，则即使帧率较高也不可采用。第二层才是**流水线收益**：在动作质量合格的配置中比较有效 FPS、stall 比例和推理耗时分布。

需要特别说明的是，在当前代码实现中触发条件为 `qsize / chunk_size <= hold`，因此 hold 越大表示越早触发推理。这与直觉中“阈值越小越早”的说法相反，后文分析均按代码语义展开。

4.3.2 实验配置与数据来源

实验采用 hold 单变量扫描。同步 baseline 复用实验 04 中无 tracing 的 A 组记录，异步模式固定 chunk_size $N = 100$ 、目标频率 $F = 30$ FPS，在同一 ACT 模型与同一 pick 任务下扫描 hold=0.10–0.60，其中 0.60 作为偏早触发的参考点。每个 hold 当前采集 1 个 60 s episode，动作质量由操作者现场观察记录；CSV 中 **success=-1** 表示本轮尚未写入机器可读成功/失败标签。

维度	取值	备注
调度模式	同步 / 异步	同步作为 baseline
chunk_size N	100	与第三章 cs=100 设置一致
目标 FPS F	30	environment_dt = 33.3 ms
hold	0.10, 0.20, 0.25, 0.30, 0.35, 0.40, 0.50, 0.60	异步调度参数
episode 时长	60 s	每个 hold 1 次
同步 baseline	16.13 FPS	实验 04 A 组无 tracing

数据文件与脚本统一放在 `lerobot/analysis/08_async_inference_hold_sweep/`：client 端输出逐 episode 的统计文件，记录动作执行帧数、stall 帧数、过期帧数、must_go 触发次数和帧率等；server 端输出逐次推理的耗时文件；此外新增 stall 事件记录，追踪每次队列耗尽到恢复的起止时间和持续帧数。离线分析时由脚本将 client 与 server 的记录按时间戳对齐合并后绘图。

4.3.3 打点与对齐方法

client 端记录的核心指标包括：执行帧数、stall 帧数、过期帧数、must_go 触发次数、帧率统计以及 CPU 温度与频率。其中有效 FPS 定义为：

$$F_{\text{eff}} = \frac{\text{action_frames}}{\text{episode_total_s}}. \quad (14)$$

该指标只统计真正执行了 action 的帧，能剔除 stall 停顿对名义循环频率的影响。

本次采集新增了 stall 事件追踪：每当控制循环发现动作队列已空时，记录当时的帧号和 episode 内时间作为 stall 起始点；当队列重新有数据可执行时记录 stall 结束点，从而得到每段连续停顿的持续时间。该信息用于判断 stall 是集中在启动阶段还是均匀分布在整个 episode。

server 端在每次推理前后分别记录时间戳，输出该次推理的耗时。本轮采集期间 server 进程持续运行，没有随 client 侧 episode 切分而重启，导致 server 日志中的 episode 编号与 client 不对齐；因此离线合并时通过检测 server 日志中帧号回跳的位置来切分各 episode 的推理记录，恢复与 client 端的对应关系。

4.3.4 实验结果

图 23 给出有效 FPS 随 hold 变化的折线。同步 baseline 为 16.13 FPS；异步各 hold 的有效 FPS 位于 19.96–27.42 FPS 之间，相对同步提升 23.8%–70.0%。图中将同步 baseline 放在 hold=0 位置作为参照点，并对 baseline–0.30 区间做经验线性拟合。需要强调的是，4.2 节的理想模型是直接由执行时间、计算时间和 stall 时间相加得到的；这里的线性拟合只对应临界点之前、一定存在 stall 的前段区间，用于描述实测点近似线性上升的趋势。从 0.10 到 0.30，FPS 随 hold 增大快速上升；0.30 和 0.35 已进入接近饱和的平台区，episode 平均 FPS 约为 27 FPS。理想模型中平台区应达到 30 FPS，实测仍有约 2–3 FPS 缺口，主要来自 episode 启动阶段的必然空队列等待，以及树莓派 5 上 robot_client、policy_server、相机线程和串口通信共享 CPU 造成的调度抖动。

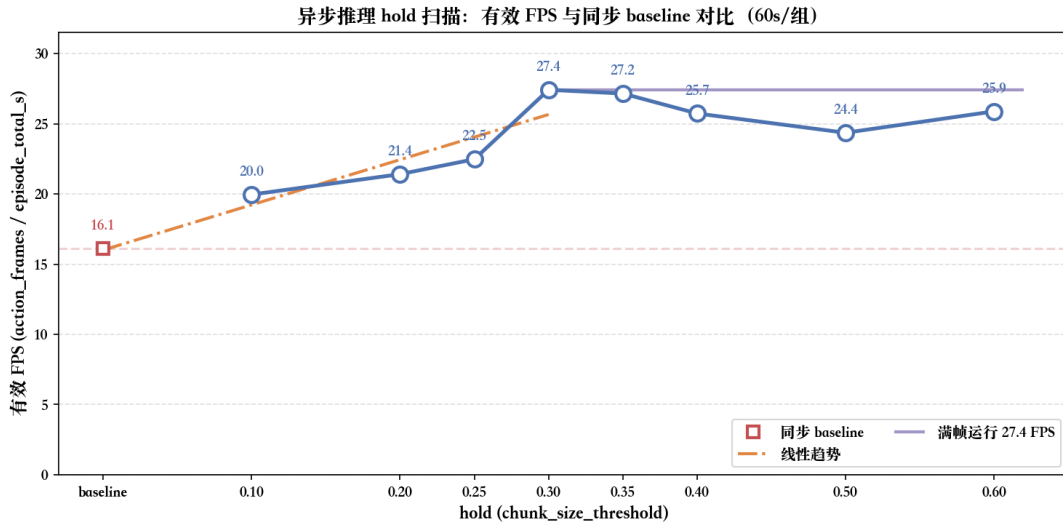


图 23: 异步推理 hold 扫描：有效 FPS、同步 baseline 与 0–0.30 经验线性拟合

表 3 汇总了核心指标。

从 stall 事件的时序分布来看，不同 hold 下的 stall 形态差异非常显著。hold=0.30 的 60 s episode 中仅出现 2 次 stall：第一次在 episode 启动时持续约 2 s（61 帧），第二次在 frame 162 附近持续约 1 s（32 帧），之后稳态阶段再无 stall。这说明 0.30 已经非常接近实际临界点：启动瞬态结束后，新 chunk 基本能在队列耗尽前返回，流

hold	动作质量	有效 FPS	相对提升	stall%	expire%
0.10	很好	19.96	23.8%	31.78	32.05
0.20	很好	21.41	32.8%	26.53	41.27
0.25	很好	22.48	39.4%	23.18	39.07
0.30	很好	27.42	70.0%	5.35	49.45
0.35	一般	27.16	68.4%	5.56	49.25
0.40	一般	25.74	59.6%	9.65	49.33
0.50	一般	24.36	51.1%	13.23	49.17
0.60	较差	25.89	60.5%	9.33	49.36

表 3: 异步推理 hold 扫描核心结果（每个 hold 1 个 60 s episode）

水线可以自持运转。

而 hold=0.25 的 stall 事件则呈现周期性：几乎每隔 4–5 s 就出现一次持续约 1 s 的 stall，全 episode 共 13 次。这说明 0.25 的触发已经偏晚，server 的推理节奏跟不上队列消耗，形成“队列耗尽 → must_go 兜底 → 新 chunk 到达 → 短暂恢复 → 再次耗尽”的周期性 stall 模式。

hold=0.50 的 stall 虽然单次持续时间更短（多为 3–10 帧，约 0.1–0.3 s），但发生频率很高，全 episode 出现 28 次。这类短 stall 不再是理论模型中的“触发太晚”，而更像平台区附近的实现误差：client 频繁采观测、server 端推理、gRPC 收发、相机后台线程和舵机串口通信都在 4 个 ARM 核心上竞争资源，使得本应覆盖住的返回时延偶尔越过队列余量。

图 24 给出执行帧、过期帧和 stall 帧的组成。随着 hold 增大，过期帧比例从 32.05% 上升至 49.45%，这与 $T_{\text{infer}}F/N \approx 1.5 \times 30/100 = 45\%$ 处于同一量级；stall 冻结 latest_action 会保护一部分未来帧不被判定为过期，因此 hold=0.10/0.20 的实测 expire% 低于直接代入的理论值。

4.3.5 结果讨论

实验结果与 4.2 节的分段模型基本一致。hold=0.10、0.20 和 0.25 仍处在临界点之前，stall 呈周期性出现，FPS 随 hold 增大而上升；hold=0.30 后周期性 stall 基本消失，说明实际临界点位于 0.25–0.30 附近。按理想模型，越过该点后稳态 FPS 应饱和到 30 FPS。实测 0.30 和 0.35 的 episode 平均 FPS 只有约 27 FPS，并不表示理论曲线在临界点后下降，而是因为 60 s episode 中仍包含启动空队列等待，并且树莓派 5 单机部署下推理进程、控制进程和 I/O 线程存在 CPU 资源争抢，使有效返回时延产生抖动。

从动作质量的角度观察，hold=0.10 至 0.30 范围内操作者均评为“很好”，0.35、

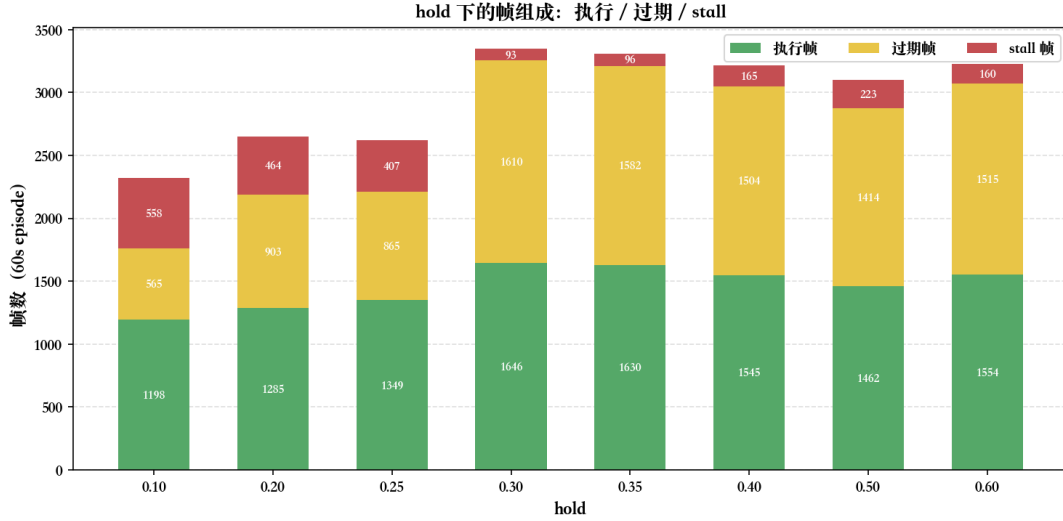


图 24: 异步推理中的执行帧、过期帧与 stall 帧组成

0.40 和 0.50 为“一般”，0.60 动作明显变差评为“较差”。这说明在本平台参数下，动作质量对 hold 的敏感度低于有效 FPS 和 stall 比例：即使 hold=0.10 的 stall 高达 31.78%，有效 FPS 仅为 19.96，temporal ensemble 仍然能在多数时刻维持轨迹连贯性。hold 继续增大后，虽然理论 FPS 已进入平台区，但更频繁的观测采集、动作融合和短 stall 会增加控制抖动，因此动作质量反而开始下降。因此在实际部署中，可以优先依据有效 FPS 和 stall 比例来选 hold，不必过度担心动作质量——只要 stall 不要极端严重（如 0.10 的 31%），动作质量仍可保持在可接受范围内。

综合有效 FPS、stall 比例、must_go 触发次数和动作质量四个维度，当前树莓派 5 + ACT 模型的最佳 hold 配置为 0.30。该配置下有效 FPS 达到 27.42，相对同步 baseline 提升 70.0%，stall 比例仅 5.35%，且 must_go 仅触发 2 次（集中在启动阶段），稳态阶段几乎无 stall，动作质量被操作者评为“很好”。0.35 的 FPS 和 stall 比例与 0.30 几乎相同，但动作质量降为“一般”，因此不推荐作为默认值。

该结论也为后续优化提供方向。若进一步通过 ONNX Runtime、量化或算子优化把有效返回时延 T_{rt} 降至 1.0 s 左右，则临界 hold 将下降到 $1.0 \times 30/100 = 0.30$ ，且 $T_{rt}F = 30 < N/2 = 50$ ，0.30 将从临界点变成有明显余量的稳定配置。若希望更晚触发、进一步减少过期帧和 temporal ensemble 冲突，则需要把 T_{rt} 继续压低；例如 hold=0.20 要进入无 stall 区，有效返回时延需接近 $0.20 \times 100/30 \approx 0.67$ s。

第五章 总结与展望

5.1 主要工作总结

本文围绕 LeRobot 在线控制中的“感知-推理-执行”闭环，研究了树莓派 5 这类无 GPU ARM 边缘平台上机器人基础软件栈的系统级性能问题。与只统计端到端 FPS 或只从模型、硬件通信单点优化入手的做法不同，本文将问题拆解到 AI 推理框架层、Python 运行时层、Linux 内核层与硬件 I/O 层，建立了面向机器人在线控制负载的四层性能剖析方法。该方法能够把上层策略调用、运行时线程行为、系统调用阻塞和底层设备通信放到同一条时间线上观察，从而为后续优化判断提供依据。

在实验环境方面，本文以 SO-101 六自由度机械臂、双摄像头输入和 ACT 策略模型为对象，构建了可复现实验平台，并围绕 `record.py` 中的核心控制循环记录各阶段耗时。在测量工具层面，本文比较了 Python 日志、`torch.profiler`、`strace` 与 `ftrace` 的适用边界。工具链校准结果表明，`strace` 会显著放大上下文切换并改变阶段时间结构，因此不适合承担定量时序分析；`ftrace` 延迟导出方案对主循环扰动相对可控，更适合作为内核态观测工具。在此基础上，本文进一步通过自定义内核插桩拆分了 `pselect6` 和 `futex` 路径中的阻塞时间与 CPU 处理时间，使用户态日志和内核态事件能够互相印证。

性能剖析结果显示，当前系统的主要瓶颈并不在单个系统调用或舵机写入函数本身，而在 ARM CPU 上执行 ACT 模型前向推理的绝对耗时。感知阶段中，两路摄像头已经通过 OpenCV 后台线程完成异步预取，主线程读取缓存帧的开销较小；执行阶段中，舵机 `sync_read` 与 `sync_write` 的毫秒级耗时主要来自 1 Mbps 半双工总线和 SCServo 协议约束，而不是 Linux 系统调用路径。相比之下，单次完整 ACT 推理耗时约为秒级，其中 ResNet18 视觉特征提取和 Transformer Encoder 占据主要比例，是限制在线控制频率的核心因素。这一结论说明，在当前硬件条件下，优化方向应优先指向推理运行时和模型算子执行效率，而不是盲目改写已受物理总线约束的硬件 I/O 路径。

在优化探索方面，本文首先分析了感知、执行与推理三类路径的异步化可行性。由于舵机读写共享同一条半双工总线，简单后台线程化会退化为“持锁串行 + 线程切换”，难以形成真正并行；相机路径已经具备后台预取机制，进一步收益有限。因此，本文将优化重点放在推理异步化上，推导了异步流水线稳定无 stall 的近似条件 $T_{\text{infer}}F \leq \min(hN, N/2)$ ，并进一步建立了 FPS 随 hold 在临界点前上升、临界点后

饱和到目标帧率的分段模型。实验结果表明，在 `chunk_size=100`、目标 30 FPS 的设置下，推荐配置 `hold=0.30` 可取得 27.42 FPS 的有效控制频率，相对同步 baseline 提升 70.0%，且稳态阶段几乎无 stall，说明异步调度能够显著回收动作执行期间的空窗。episode 平均 FPS 尚未达到 30 的主要原因是启动空队列等待和树莓派 5 单机部署下的 CPU 资源争抢，后续仍需从机器人基础软件栈中的推理运行时、操作系统调度和跨层诊断工具继续优化。

5.2 研究局限性

本文的实验对象固定在树莓派 5、SO-101 机械臂和 ACT 策略模型上，因此结论首先服务于 ARM CPU 边缘机器人这一类受限平台。不同机械臂的通信协议、相机驱动、策略模型结构和任务动作尺度可能改变各阶段占比，尤其是在模型更小、舵机总线更慢或相机分辨率更高的配置中，瓶颈可能从推理运行时转移到硬件 I/O 或图像预处理路径。因此，本文得到的具体数值不应直接外推到所有机器人平台，更适合作为一套跨层性能分析方法和基础软件优化思路的案例验证。

其次，本文已经对异步推理进行了实测试验，但对推理运行时本体的优化仍然有限。ONNX Runtime、INT8 量化、算子融合、ARM NEON 后端优化以及 PyTorch 线程池参数等方向，在本文中主要作为瓶颈定位后的推论和未来工作提出，尚未形成完整的对比实验。因此，本文能够说明当前系统为什么慢、异步调度能恢复多少吞吐，但还不能给出降低单次 ACT 推理耗时的最终工程方案。

最后，本文的动作质量评价仍以操作者现场观察为主，尚未建立独立于主观判断的任务质量指标。异步推理中的 stall 比例、过期帧比例和有效 FPS 可以反映控制流水线的运行状态，但它们并不能完全替代抓取成功率、末端轨迹误差、动作平滑度和安全裕量等机器人任务指标。后续若要将本文方法推广为更通用的基础软件评测框架，仍需把系统级性能指标与任务级行为质量进一步关联起来。

5.3 未来工作

未来工作应继续围绕机器人基础软件栈展开，而不是停留在简单更换任务或平台的层面。首先，在推理运行时方面，需要针对 ARM CPU 上 ACT 推理耗时过高的问题开展系统优化实验。可优先比较 PyTorch eager 模式、ONNX Runtime、INT8 量化和算子融合方案在树莓派 5 上的端到端收益，并进一步分析 ResNet18 卷积、Transformer 线性层和 GEMM 算子在 NEON 或 KleidiAI 等 ARM 后端上的执行效率。同时，还需要评估 PyTorch 线程池规模、OpenMP 调度策略和模型 batch/shape

固定化对延迟抖动的影响，使推理运行时优化能够服务于稳定控制频率，而不仅是单次 benchmark 的峰值速度。

其次，在操作系统与调度层面，可以继续研究 Linux 对机器人在线控制负载的支持能力。当前异步推理实验中，`policy_server`、`robot_client`、相机线程和串口通信都竞争同一组 CPU 核心，因而推理耗时和控制循环 jitter 会受到调度策略影响。后续可围绕 CPU 频率锁定、线程亲和性、实时优先级、`cgroup` 资源隔离和 `PREEMPT_RT` 内核展开对比，量化这些机制对 `futex` 等待、上下文切换、队列 stall 和有效 FPS 的影响。这类实验有助于回答机器人基础软件中一个更本质的问题：在有限 CPU 资源上，推理任务、控制任务和 I/O 任务应如何被操作系统共同调度。

再次，在硬件 I/O 基础库方面，仍需区分“可以被软件隐藏的等待”和“由协议物理约束决定的等待”。摄像头路径可继续从 V4L2 缓冲队列、OpenCV 后台线程、MJPEG 解码和图像张量转换等层面分析端到端时延，明确后台预取在不同帧率与分辨率下的边界。舵机路径则应围绕 SCServo 半双工总线、串口波特率、同步读写协议和潜在的流水线读机制继续建模，判断哪些优化只是移动阻塞位置，哪些优化能够真正减少主控制路径耗时。这种面向 I/O 基础库的分析对机器人系统尤其重要，因为硬件通信往往同时涉及性能、安全和动作语义，不能只按普通服务器程序中的异步 I/O 经验处理。

最后，本文使用的跨层 profiling 方法仍可进一步工具化。后续可以将 Python 日志、`torch.profiler`、`ftrace`、内核自定义插桩和实验 CSV 自动对齐为统一时间线，形成面向机器人基础软件的可复用性能诊断工具。该工具应能够自动标注每一帧中的感知、推理、执行和等待区间，关联用户态函数、内核阻塞点和硬件设备事件，并输出适合优化决策的指标，例如推理临界时间、stall 触发原因、I/O 等待占比和调度抖动来源。通过这种方式，本文的工作可以从一次针对 LeRobot 的系统剖析，进一步发展为面向边缘机器人基础软件栈的通用评测与优化方法。

参考文献

- [1] ZHAO T Z, KUMAR V, LEVINE S, et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.
- [2] CHI C, FENG S, DU Y, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.
- [3] CADENE R, ALIBERT S, et al. LeRobot[EB/OL]. 2024. <https://github.com/huggingface/lerobot>.
- [4] GUTIÉRREZ C S V, JUAN L U S, UGARTE I Z, et al. Real-time Linux Communications: An Evaluation of the Linux Communication Stack for Real-Time Robotic Applications[J]. arXiv preprint arXiv:1808.10821, 2018.
- [5] A Survey of Real-Time Support, Analysis, and Advancements in ROS 2[J]. arXiv preprint arXiv:2601.10722, 2026.
- [6] ZHENG W, LI B, XU B, et al. Leveraging OS-Level Primitives for Robotic Action Management[J]. arXiv preprint arXiv:2508.10259, 2025.
- [7] MACENSKI S, FOOTE T, GERKEY B, et al. Robot Operating System 2: Design, architecture, and uses in the wild[C]//Science Robotics: vol. 7: 66. 2022.
- [8] SOKEN-HUBERTY E, et al. Impact of ROS 2 Node Composition in Robotic Systems[J]. arXiv preprint arXiv:2305.09933, 2023.
- [9] ASWANI P, et al. Performance Evaluation of ROS2-DDS Middleware Implementations Facilitating Cooperative Driving in Autonomous Vehicle[J]. arXiv preprint arXiv:2412.07485, 2024.
- [10] BÉDARD C, LÜTKEBOHLE I, DAGENAIS M. ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2[J]. arXiv preprint arXiv:2201.00393, 2022.
- [11] Google. TensorFlow Lite[J/OL]. 2024. <https://www.tensorflow.org/lite>.
- [12] Microsoft. ONNX Runtime[J/OL]. 2024. <https://github.com/microsoft/onnxruntime>.

- [13] BROHAN A, BROWN N, CARBAJAL J, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[J]. arXiv preprint arXiv:2307.15818, 2023.
- [14] KIM M J, PUMACAY K, JATAVALLABHULA K M, et al. OpenVLA: An Open-Source Vision-Language-Action Model[J]. arXiv preprint arXiv:2406.09246, 2024.

附 录

附录 A：待补充实验清单

优先级	实验内容	填充章节	预计耗时
★★★	推倒/分类任务计时数据	第 3.2.2 节三类任务对比	已完成
★★★	torch.profiler op-level	第 3.2.3 节算子分解	1 天
★★★	摄像头层级耗时剖析（实验 06）	第 3.1.2 节 Call-Stack 源码分析	1 天
★★	CPU 频率锁定对比	第 4.2 节峰值成因	半天
★	perf stat (IPC, cache-miss)	判断 compute vs memory bound	半天

致 谢

[TODO: 致谢内容]